

Smart-Host Analytics

David Menéndez Rodríguez

IES Azarquiel

alumno.1431653@ies-azarquiel.es

Abril 9, 2026

ABSTRACT

El auge de las plataformas de alquiler vacacional como Airbnb ha transformado el mercado inmobiliario turístico, dificultando a los inversores la tasación precisa de activos debido a la "ceguera estética" de los Modelos de Valoración Automatizada (AVMs) tradicionales. Este proyecto presenta SmartHost Analytics, un Sistema de Soporte a Decisiones Financieras orientado a inversores en la ciudad de Málaga, capaz de predecir el impacto económico directo del diseño de interiores sobre los ingresos anuales esperados.

El sistema implementa una arquitectura híbrida de *Deep Learning* multimodal mediante *Late Fusion*. Por un lado, una rama densa (MLP) procesa características espaciales de proximidad (*Haversine*) y estructurales del inmueble, estableciendo un precio base ($R^2=0.57$). Por otro, una rama visual basada en *Transfer Learning* (ResNet34) extrae *embeddings* de las fotografías de portada para ponderar su calidad y modernidad. La concatenación de ambos vectores arroja un **R^2 de 0.692** y reduce el Error Absoluto Medio (MAE) a **21,47 €/noche**.

Desplegado bajo una arquitectura de microservicios con FastAPI (Backend) y Streamlit (Frontend), SmartHost Analytics permite a los inversores simular dinámicamente el Retorno de Inversión (ROI) de futuras reformas, demostrando que la Inteligencia Artificial no solo tasa activos, sino que optimiza decisiones de capital en base a la estética percibida.

KEYWORDS

Machine Learning, Redes Neuronales Multimodales, Transfer Learning, Visión Artificial, ResNet34, Real Estate, SmartHost Analytics, Retorno de Inversión (ROI), FastAPI, Streamlit.

1. INTRODUCCIÓN Y DEFINICIÓN DEL PROBLEMA

1.1. Contexto del Mercado Inmobiliario Turístico

En los últimos años, la ciudad de Málaga, como otras ciudades de España a las que también podría aplicarse este modelo, se han consolidado como uno de los epicentros de inversión en apartamentos turísticos a nivel europeo. En este ecosistema altamente competitivo, operado principalmente a través de plataformas como Airbnb, los inversores inmobiliarios se enfrentan a un desafío constante: Maximizar la rentabilidad de sus activos. La decisión de adquirir una propiedad suele ir acompañada del dilema de la reforma; sin embargo, hasta ahora, el impacto económico de la decoración y el mobiliario se ha basado en la intuición y la subjetividad del inversor, careciendo de un respaldo analítico.

1.2. Definición del Problema: La "Ceguera" de las Tasaciones Actuales

Los modelos de valoración automatizada tradicionales (*Automated Valuation Models* o AVMs) y las herramientas de análisis de mercado actuales presentan una limitación crítica: son "ciegos" a la estética. Estos sistemas calculan el precio base de mercado basándose exclusivamente en características físicas y tabulares estáticas (ubicación, número de habitaciones, baños o servicios).

Sin embargo, en el mercado vacacional, el principal motor de conversión (clics y reservas) es el **atractivo visual** de la foto de portada. Un apartamento con mobiliario anticuado y mala iluminación no generará los mismos ingresos que un apartamento idéntico, en el mismo edificio, que haya sido sometido a un proyecto de interiorismo moderno. Actualmente no existe en el mercado una herramienta accesible que cuantifique matemáticamente este "atractivo visual" y lo traduzca en una previsión de facturación. Esto provoca que los inversores asuman un riesgo ciego, gastando miles de euros en reformas sin poder prever si el mercado realmente absorberá esa subida de precio o si, por el contrario, será un coste irrecuperable o superfluo.

1.3. La Solución: SmartHost Analytics y su Valor de Negocio

Para resolver esta deficiencia tecnológica, nace **SmartHost Analytics**, una herramienta de Inteligencia Artificial diseñada específicamente para inversores inmobiliarios en Málaga. El objetivo principal de este proyecto es desarrollar un sistema de tasación avanzado utilizando redes neuronales multimodales.

La innovación del sistema radica en su arquitectura dual:

1. Por un lado, procesa los datos numéricos y categóricos para establecer un precio base competitivo.
2. Por otro, evalúa el "atractivo visual" de la foto de portada del inmueble mediante técnicas de Visión Artificial, imitando la percepción del huésped potencial.
3. Estos dos análisis por separado, posteriormente se ponen en común y se fusionan.

Valor de Negocio y ROI: La fusión de ambas redes permite predecir de forma precisa cómo la estética impactará en los ingresos finales. El gran valor aportado al sector es la capacidad de simulación: un inversor puede introducir los datos de una vivienda a reformar y subir fotografías de inspiración (por ejemplo, renders o catálogos de IKEA). La Inteligencia Artificial recalculará instantáneamente los ingresos anuales esperados, permitiendo medir el Retorno de Inversión (ROI) de la reforma antes de gastar un solo euro en la obra. De este modo, la toma de decisiones basada en emociones se transforma en una estrategia pura de datos.

2. ESTADO DEL ARTE

2.1. Modelos de Valoración Automatizada Tradicionales (AVMs) y su Limitación Estructural

El sector inmobiliario ha dependido históricamente de los Modelos de Valoración Automatizada (*Automated Valuation Models* o AVMs) para estimar el valor de mercado de los activos de forma sistemática. Estos modelos analizan masivamente datos de carácter puramente tabular y estático, tales como la superficie construida, el número de dormitorios, la cantidad de cuartos de baño, el año de edificación y la delimitación por códigos postales o distritos censales. En sus inicios, los AVMs operaban mediante regresiones lineales múltiples; posteriormente, evolucionaron hacia modelos basados en árboles de decisión y algoritmos de *gradient boosting* (como XGBoost o Random Forest), logrando capturar relaciones no lineales complejas entre las características físicas de la vivienda y su precio de venta o alquiler.

Sin embargo, el auge de las plataformas de economía colaborativa y alquiler vacacional (como Airbnb o Booking.com) ha introducido dinámicas comerciales que escapan por completo al alcance de las variables tabulares clásicas. En el mercado del alojamiento turístico, el comportamiento del consumidor está fuertemente condicionado por factores heurísticos y emocionales, donde la primera impresión visual actúa como el principal motor de conversión (tasa de clics o *Click-Through Rate* y volumen de reservas masivas).

Es en este escenario donde los AVMs tradicionales exhiben una limitación crítica conocida como la **"ceguera estética"**. Dos propiedades ubicadas en el mismo edificio, con idéntica orientación, superficie y número de habitaciones, recibirán la misma tasación por parte de un AVM convencional. No obstante, si una de ellas cuenta con un diseño de interiorismo moderno, iluminación optimizada y mobiliario de alta calidad, y la otra presenta decoración obsoleta y malas condiciones lumínicas, sus ingresos reales por noche diferirán drásticamente. Los modelos tradicionales son incapaces de ponderar el impacto económico de la calidad percibida, lo que obliga a los inversores a depender de su intuición subjetiva a la hora de presupuestar reformas estéticas.

2.2. Evolución hacia el Aprendizaje Profundo (*Deep Learning*) en Real Estate

Para superar las barreras de los modelos predictivos tradicionales, la investigación contemporánea se ha desplazado hacia el uso de Redes Neuronal Profundas (DNN). El Perceptrón Multicapa (*Multi-Layer Perceptron* o MLP) ha demostrado una capacidad superior para procesar grandes volúmenes de datos ruidosos y asimétricos, abstrayendo patrones de alta dimensionalidad mediante capas ocultas con funciones de activación no lineales como ReLU.

En el ámbito de la ingeniería de características (*Feature Engineering*) para Real Estate, las DNN permiten integrar variables dinámicas enriquecidas. En lugar de utilizar clasificaciones geográficas rígidas (como el nombre del barrio), las redes profundas asimilan variables de proximidad continua calculadas mediante trigonometría esférica, como la fórmula del semiverseno (*Haversine*), para determinar distancias exactas a puntos de alto interés turístico (costa, playas, centros financieros o históricos). Esto dota al modelo tabular de una sensibilidad espacial muy superior, permitiendo mapear con precisión micro-mercados dentro de una misma urbe.

2.3. Modelado Multimodal y Visión Artificial: La Frontera del Conocimiento

La verdadera frontera científica en la valoración de activos radica en el **Modelado Multimodal**, una de las disciplinas más avanzadas del Aprendizaje Profundo que busca unificar distintas fuentes de información (texto, datos numéricos e imágenes) en un único espacio latente de toma de decisiones.

El procesamiento de la información visual ha experimentado una revolución gracias a las Redes Neuronales Convolucionales (CNN) y al auge del Aprendizaje por Transferencia (*Transfer Learning*). Investigaciones fundamentales en el área del diseño de redes, con la introducción de las redes residuales (**ResNet**), demostraron que es posible aprovechar modelos profundamente entrenados en bancos de imágenes masivos (como ImageNet) para extraer descriptores visuales de alta fidelidad (texturas, paletas de colores, distribución espacial y luminosidad) sin incurrir en un coste de computación inasumible.

En la literatura científica reciente sobre tasación inmobiliaria, los enfoques multimodales basados en la técnica de **Fusión Tardía (*Late Fusion*)** han demostrado batir sistemáticamente los récords de precisión de cualquier modelo unidimensional. El

procedimiento estándar consiste en truncar la capa de clasificación de una CNN preentrenada (como ResNet34) para transformarla en un extractor puro de características, obteniendo un vector de *embeddings* visuales que resume matemáticamente la calidad de la imagen. Posteriormente, mediante una operación de concatenación en PyTorch, este vector visual se unifica con el vector de características tabulares procesado de forma paralela por una rama MLP.

2.4. Justificación de la Innovación de SmartHost Analytics

A pesar de la sólida base teórica de las redes multimodales en el ámbito académico, la práctica totalidad de las soluciones comerciales disponibles en el mercado actual continúan ignorando la información visual, o bien la limitan a clasificaciones binarias rudimentarias.

La innovación diferencial de **SmartHost Analytics** dentro del estado del arte no radica únicamente en el acoplamiento de una arquitectura ResNet34 y una red densa MLP para reducir el Error Absoluto Medio (MAE) de tasación, sino en su **orientación funcional hacia la lógica de negocio y la simulación interactiva**. El sistema rompe el paradigma de la tasación estática para convertirse en un **Sistema de Soporte a Decisiones Financieras**. Al permitir al usuario subir imágenes de inspiración o renders estéticos futuros (procedentes de catálogos o proyectos de interiorismo) y cruzar dicha información con ratios dinámicos de ocupación anual corregida, la plataforma es capaz de aislar matemáticamente el impacto del "atractivo visual" y estimar con precisión el Retorno de Inversión (ROI) de una reforma antes de que el inversor ejecute el gasto de capital. Esto posiciona al proyecto como una solución pionera y disruptiva, que cubre una brecha tecnológica real tanto en el tejido empresarial como en el entorno académico vacacional.

3. TRABAJO REALIZADO

3.1. Preprocesamiento y limpieza

3.1.1. Fuente de Datos y Extracción de Imágenes

El conjunto de datos tabulares utilizado en este proyecto proviene de **Inside Airbnb**, una plataforma independiente que publica periódicamente datos extraídos de los anuncios públicos de Airbnb para distintas ciudades. El dataset de partida para Málaga comprende **9.714 registros y 79 columnas**, incluyendo características físicas del alojamiento, información del anfitrión, métricas de valoración de huéspedes y el precio base por noche.

Dado que el modelo multimodal requiere no solo datos tabulares sino también las imágenes de portada de cada anuncio, se desarrolló un **script de extracción programática** (`script_imagenes_portada.py`) que automatiza la descarga de dichas imágenes directamente desde los servidores de Airbnb. El script lee la columna `picture_url` del dataset, que contiene el enlace directo a la fotografía principal de cada alojamiento, y descarga cada imagen nombrándola con el id del alojamiento, garantizando así la trazabilidad entre el dato tabular y su imagen correspondiente durante el entrenamiento. El proceso, después de una primera limpieza básica del dataset, permitió obtener aproximadamente **6800 imágenes** válidas, suficientes para el entrenamiento de la rama convolucional del modelo. Los anuncios cuya URL de imagen había quedado desactualizada —aproximadamente un 5% del total— fueron identificados cruzando los identificadores del dataset con los archivos presentes en disco y eliminados del conjunto de datos en una fase posterior de limpieza.

3.1.2. Limpieza del Dataset Tabular

Una vez cargado el dataset original, se llevó a cabo un proceso de limpieza estructurado en varias etapas:

Conversión de tipos. La columna `price`, que se encontraba en formato texto, se limpia y convierte a numérica, también se limpian otras columnas como porcentajes.

Extracción de variables desde texto. La columna `bathrooms_text`, que describía los baños en lenguaje natural (ej. "1.5 shared baths"), fue procesada en dos pasos: primero se creó una variable binaria `private_bathroom` que toma valor 1 si el baño es privado y 0 si es compartido, detectando la presencia de la cadena "shared" en el texto; posteriormente se extrajo el valor numérico mediante expresión regular, generando la columna `bathrooms`.

Eliminación de anuncios inactivos o no representativos. En primer lugar, los denominados pisos fantasma: Anuncios con menos de 3 valoraciones históricas ($\text{number_of_reviews} < 3$). En segundo lugar, los anuncios sin actividad reciente: Para considerar un anuncio activo, se estableció que debía cumplir al menos una de las siguientes condiciones: haber estado ocupado un mínimo de 30 días en los últimos 365 días ($\text{estimated_occupancy_l365d} \geq 30$), haber recibido al menos 3 reseñas en los últimos 12 meses ($\text{number_of_reviews_ltm} \geq 3$), o al menos 3 reseñas en el último año natural ($\text{number_of_reviews_ly} \geq 3$). Tras aplicar estos filtros, el dataset quedó reducido a **5.796 registros**.

Reducción de dimensionalidad. De las 79 columnas originales se eliminaron aquellas sin valor predictivo demostrado: identificadores y URLs de texto libre, columnas redundantes, variables del anfitrión no significativas y duplicados en las métricas de reseñas. También se eliminan variables con influencia prácticamente nula en el precio, mediante matriz de correlación. El dataset final de trabajo quedó compuesto por **47 columnas**.

3.1.3. Ingeniería de Características

A partir de las variables disponibles se busca construir nuevas columnas con mayor capacidad explicativa para el modelo:

Distancia geográfica al centro. Se calculó la distancia en kilómetros desde cada alojamiento hasta el centro histórico de Málaga (Calle Larios) mediante la fórmula de Haversine, que proporciona la distancia de círculo máximo sobre la superficie terrestre con mayor precisión que la distancia euclidiana plana.

Distancia a la playa más cercana. Aplicando la misma fórmula de Haversine, se calculó la distancia mínima de cada alojamiento a un catálogo de 19 playas del litoral malagueño, desde El Cañuelo en el límite oeste hasta El Hornillo en el límite este.

Índice de hacinamiento. Calculado como el número de huéspedes por habitación disponible, se divide el número de ocupantes entre las habitaciones del apartamento.

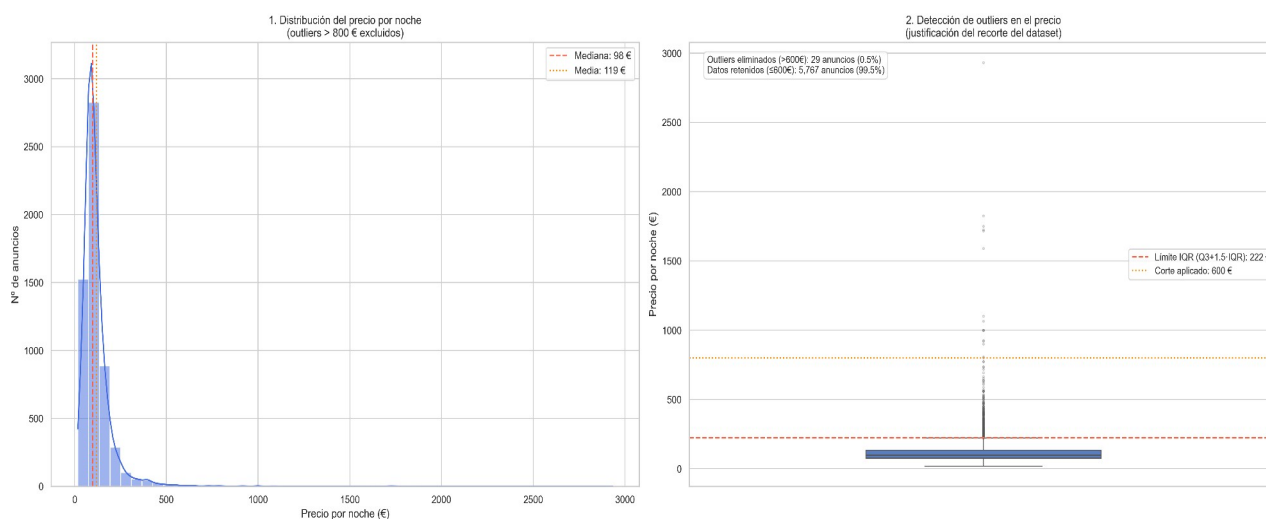
Ratio de baños por huésped. Indicador de confort sanitario del alojamiento: Se dividen los ocupantes entre el número de baños.

Precio por persona. Dado que la correlación entre price y accommodates es de 0,52, el precio bruto está fuertemente condicionado por la capacidad. Esta variable normaliza dicho efecto: Se divide el precio entre los ocupantes del airbnb

Puntuación de servicios (*amenities score*). Suma del total de servicios presentes en el alojamiento, calculada a partir de las variables binarias `has_*` generadas mediante One-Hot Encoding sobre la columna original `amenities`. Proporciona al modelo una señal compacta de la riqueza de equipamiento del alojamiento.

3.2. Análisis Exploratorio de Datos (EDA)

3.2.1. Tratamiento de outliers en el target



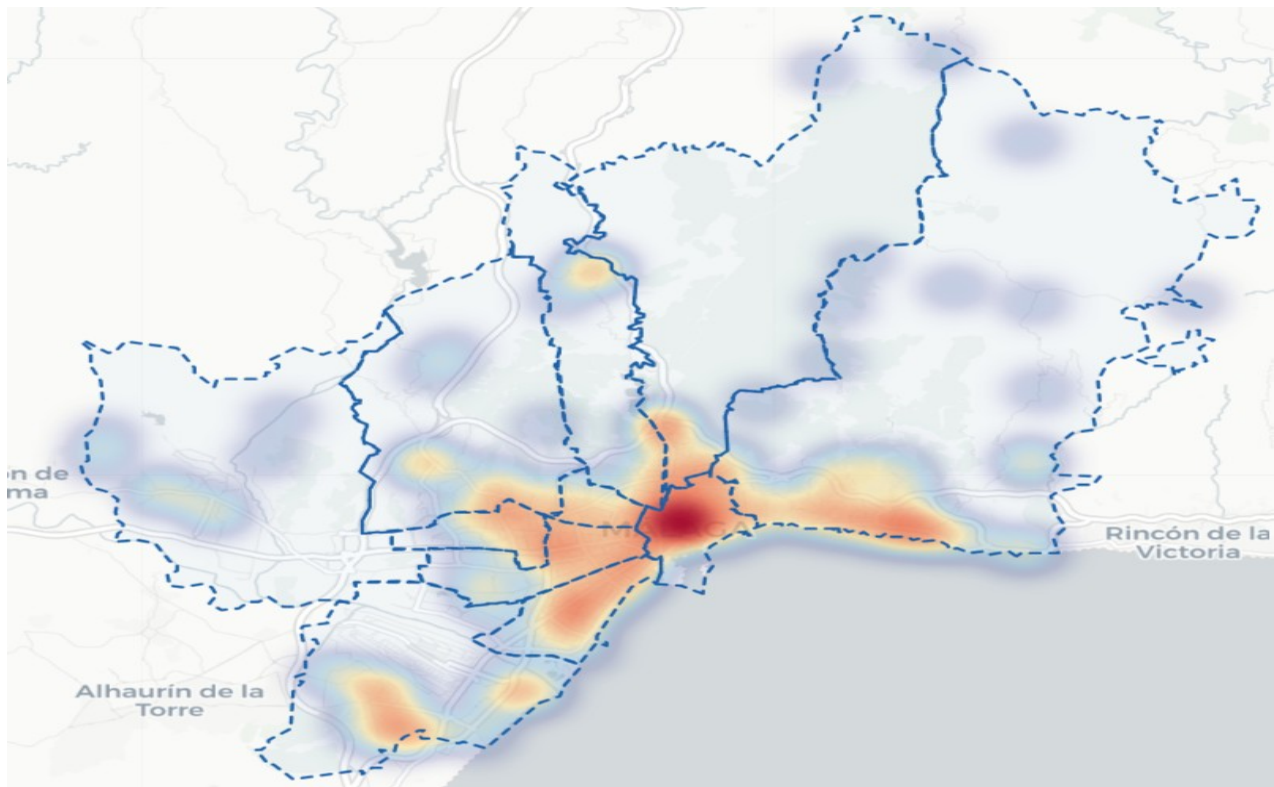
Como muestra el histograma de la figura anterior, la **variable price presenta una asimetría positiva pronunciada**: la gran mayoría de los anuncios se concentra entre 50 y 300 €/noche, pero una minoría (outliers) de registros alcanza valores de 1.500, 2.000 e incluso 3.000 €/noche, presuntamente propiedades de lujo o villas de alto standing con escasa representación en el dataset. Este fenómeno tiene dos consecuencias directas:

Distorsión visual: Al representar el precio en escala bruta, estos pocos valores extremos estiran el eje horizontal hasta los 3.000 €, aplastando el 99% de los datos en el margen izquierdo e impidiendo extraer conclusiones visuales sobre el mercado real.

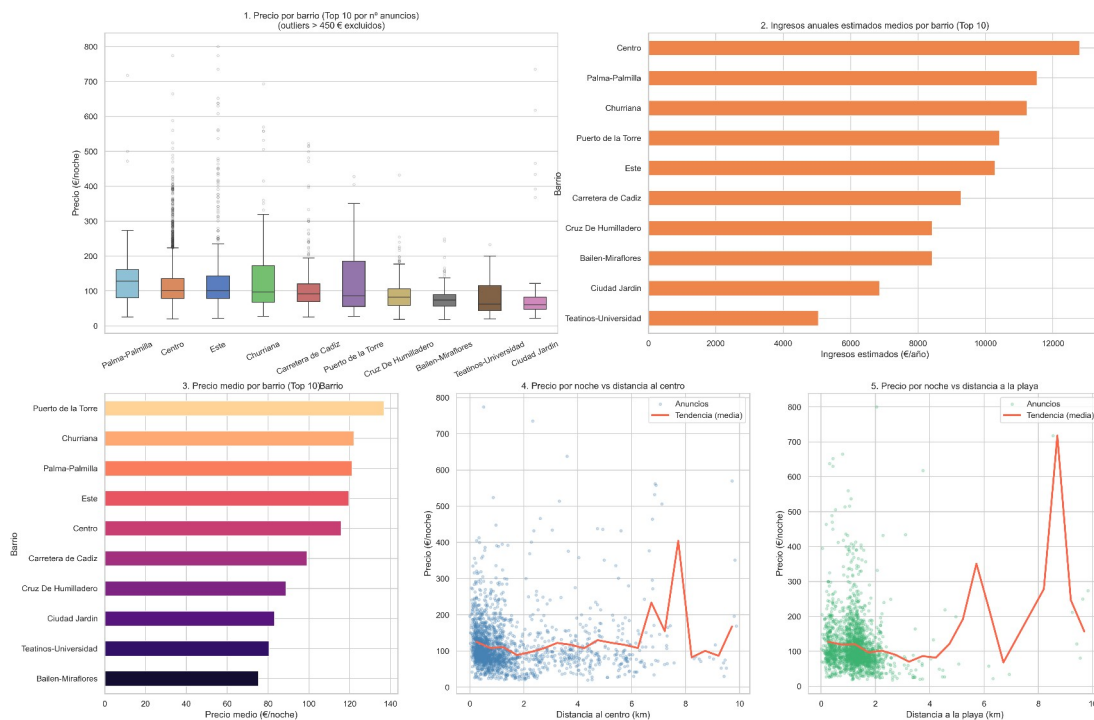
Riesgo para el entrenamiento: Si se entrena la red neuronal con estos valores extremos sin control, el modelo podría sobreponderar dichos casos durante la optimización, degradando su rendimiento sobre el segmento mayoritario del mercado.

Se estableció el corte en 600 €/noche. Con este umbral, únicamente se eliminaron 29 anuncios (0,5%), reteniendo 5.767 registros (99,5%) que representan fielmente la realidad del mercado turístico malagueño. La diferencia entre la mediana (98 €) y la media (119 €) una brecha de 21 € es en sí misma un indicador cuantificable del grado de asimetría residual que persiste incluso tras el filtrado. Dado que el Error Cuadrático Medio (MSE) eleva al cuadrado los errores grandes y penaliza desproporcionadamente los outliers, se optará por el ****Error Absoluto Medio (MAE)**** como métrica principal, al ser más robusto frente a esta distribución asimétrica del target.

3.2.2. Geografía y precio: El mercado dual de Málaga



El mapa de calor revela que la oferta de alojamiento turístico no se distribuye de forma uniforme sobre el territorio, sino que se concentra en dos focos bien diferenciados: el núcleo del centro histórico y el corredor litoral entre La Malagueta y El Palo. Fuera de este eje urbano-costero, la densidad cae drásticamente. Este patrón responde al perfil del turista tipo en Málaga, que busca simultáneamente la proximidad al patrimonio cultural (Alcazaba, Catedral, Calle Larios) y el acceso al mar. La consecuencia directa es que la ubicación es, a priori, la variable con mayor poder predictivo sobre el precio, lo que motivó el cálculo de las variables `distancia_centro_km` y `distancia_playa_km`.



El análisis por barrios revela una paradoja aparente: el Centro no lidera el ranking de precio medio por noche, siendo superado por barrios periféricos como Puerto de la Torre, Churriana o Palma-Palmilla. Sin embargo, el gráfico de ingresos anuales estimados muestra la imagen inversa: el Centro es con diferencia el barrio más rentable en términos anuales. Esto demuestra que precio por noche y rentabilidad anual son variables con dinámicas distintas; el Centro cobra precios moderados pero mantiene tasas de ocupación muy superiores al resto, lo que justifica que SmarHost Analytics prediga ambos targets de forma independiente.

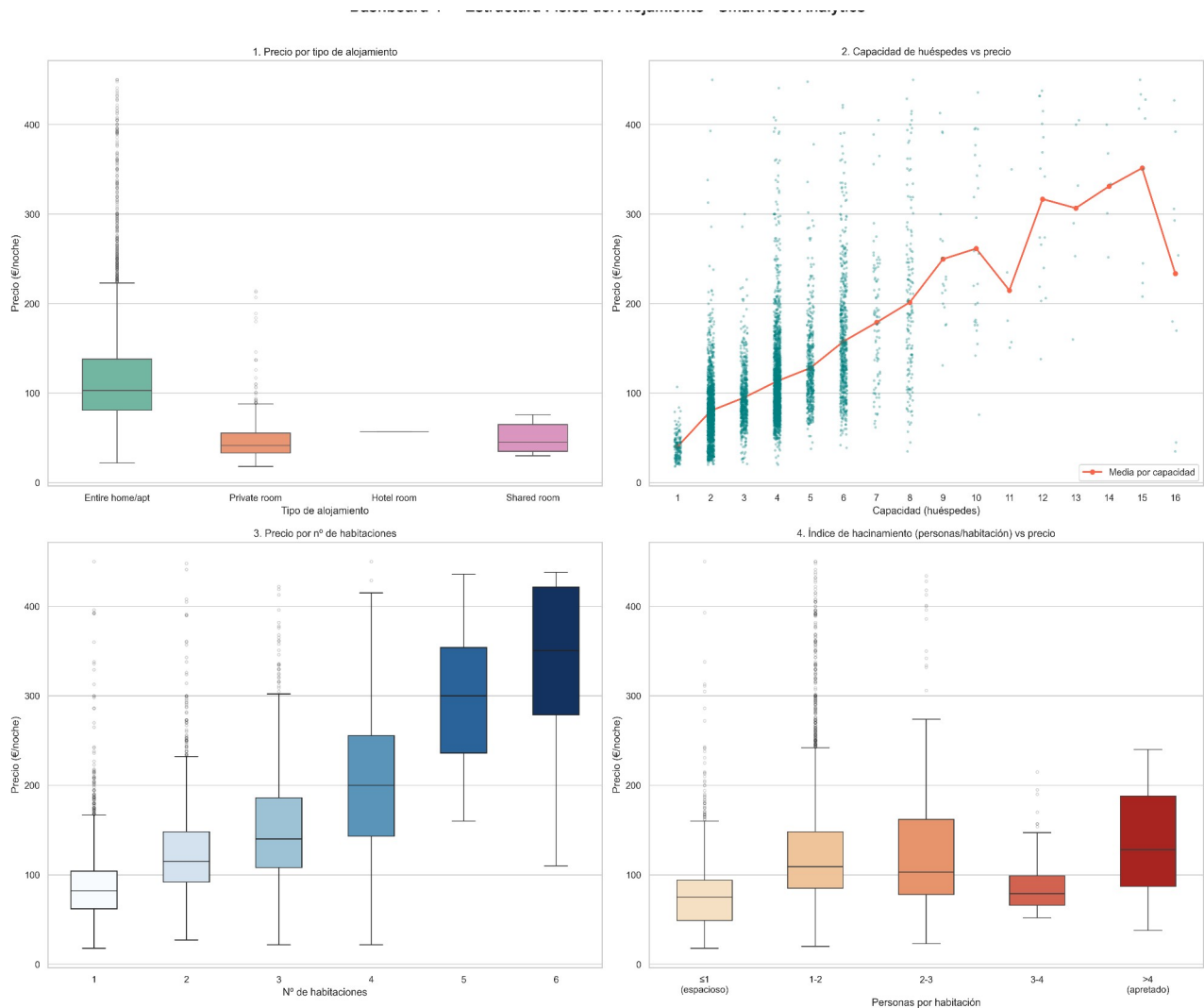
El boxplot de precios por barrio (gráfico 1) muestra además que el Centro presenta una dispersión interna muy elevada, con una mediana contenida pero una cola derecha de outliers que alcanza los 400-500 €/noche. Esto evidencia la coexistencia de dos sub-mercados dentro del mismo barrio: habitaciones privadas, estudios pequeños y apartamentos de lujo o áticos en ubicaciones privilegiadas. La red neuronal deberá aprender a separar estos perfiles.

Por su parte, los scatter plots de distancia (gráficos 4 y 5) revelan que la relación entre precio y distancia no es lineal sino que adopta una forma de U asimétrica: el precio desciende moderadamente al alejarse del centro en los primeros 4-5 km, correspondientes a zonas residenciales estándar, pero repunta con fuerza a partir de los

6-7 km, donde aparecen villas, chalets y casas rurales de alta capacidad que justifican un precio por noche elevado. Este mismo patrón se reproduce para la distancia a la playa.

3.2.3. La dimensión física y los ratios de confort

El análisis de la estructura física del alojamiento confirma que el tamaño es el grupo de variables con mayor capacidad predictiva lineal sobre el precio, y permite además validar la utilidad de los ratios de confort construidos durante la fase de ingeniería de características.



El gráfico 1 evidencia la brecha existente entre el alojamiento completo (Entire home/apt) y el resto de categorías, con mediana incluso al doble de precio que el resto, cuyo rango intercuartílico se extiende desde los 75 hasta los 140 €/noche con una larga cola superior de outliers. Demostrando que `room_type` por sí sola es insuficiente para predecir el precio; necesita combinarse con otras variables como la ubicación, capacidad y los servicios. En consecuencia, `room_type` se incorporará al modelo como variable categórica codificada, esperándose que sea una de las features con mayor peso en la red.

El gráfico 2 muestra la relación entre capacidad de huéspedes y precio. La tendencia media es claramente creciente y aproximadamente lineal entre 1 y 8 huéspedes. A partir de esa capacidad, el comportamiento se vuelve irregular debido a la escasez de muestras en ese rango. Se confirma por tanto como un predictor numérico robusto para el segmento central del mercado.

La progresión más ordenada y consistente del dashboard es la del gráfico 3, que muestra el precio por número de habitaciones. Cada habitación adicional eleva la mediana de forma clara y sin excepciones. Esta relación monótona y casi lineal es exactamente el tipo de señal que las redes neuronales aprenden de forma eficiente, sitúa a bedrooms junto a accommodates como el núcleo de la dimensión física del alojamiento.

El gráfico 4, correspondiente al índice de hacinamiento (personas por habitación), arroja un resultado contraintuitivo: las medianas de precio son similares entre los rangos 1-2, 2-3 y 3-4 personas por habitación, sin la penalización esperada a medida que aumenta el hacinamiento. Una posible hipótesis puede ser que esta variable no distingue el tipo de propiedad: un piso entero de una habitación con cuatro huéspedes tiene un índice de hacinamiento alto pero puede cobrar perfectamente 120 €/noche por ubicación o servicios. El índice de hacinamiento no muestra, por tanto, una relación directa y aislada con el precio, lo que no invalida su inclusión en el modelo, pero sí descarta que sea un predictor fuerte de forma independiente.

3.2.4. Servicios del alojamiento - Amenities

El análisis de los servicios del alojamiento tiene como objetivo validar la utilidad del `amenities_score` construido durante la fase de ingeniería de características, así como identificar qué servicios concretos tienen mayor impacto económico sobre el precio.



El gráfico 1 muestra que los servicios más prevalentes del dataset son el aire acondicionado (~97%), la cocina, la TV, la lavadora, el secador y la plancha. Estos servicios han dejado de ser diferenciadores en el mercado de alquiler vacacional para convertirse en requisitos mínimos de entrada: su ausencia penaliza el precio, pero su presencia apenas discrimina entre anuncios. En cambio, terraza o balcón (~40%) es el único servicio con carácter aspiracional real, lo que explica por qué, a pesar de su baja frecuencia, tiene un impacto económico significativo como se verá en el gráfico 4.

Los gráficos 2 y 3 confirman desde perspectivas complementarias que a mayor `amenities_score`, mayor precio. El scatter muestra una tendencia media claramente

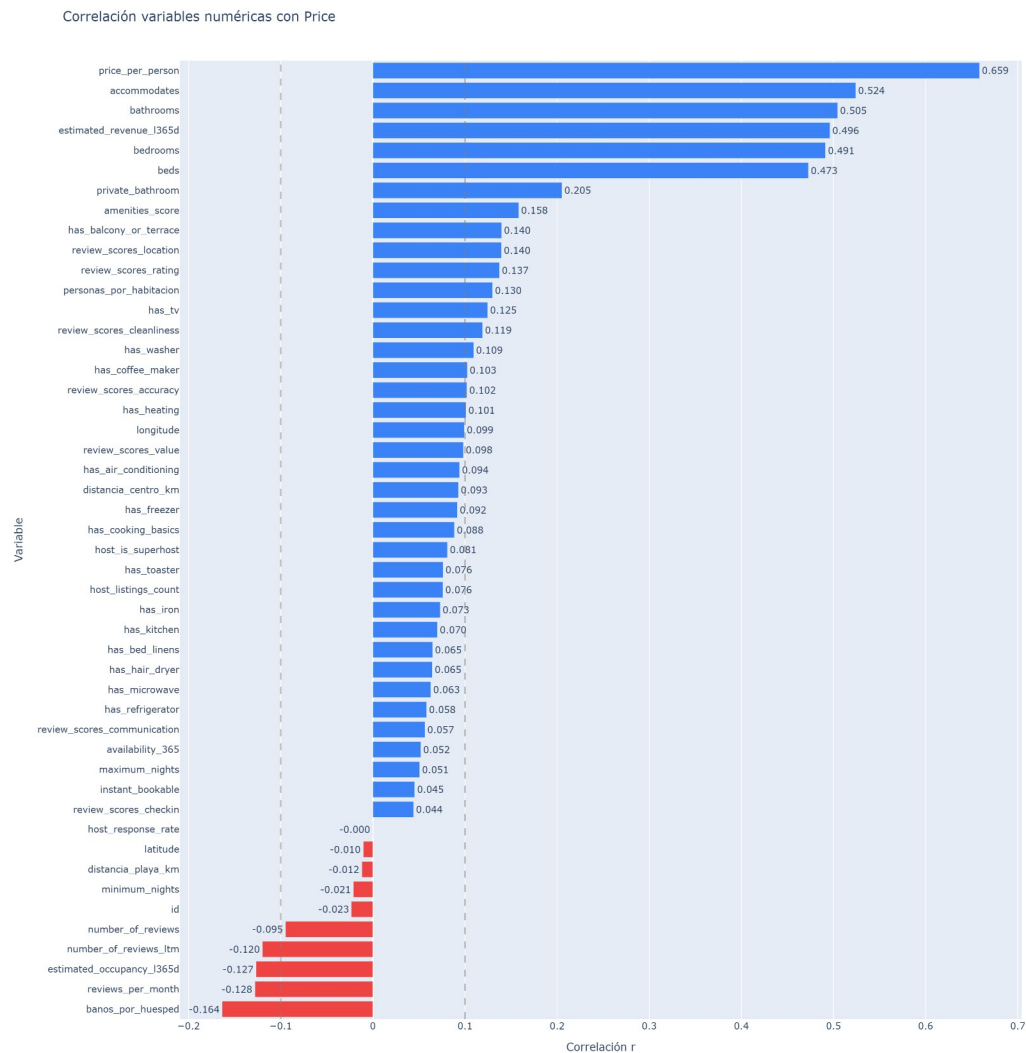
creciente a medida que se acumula el score, con el suelo y el techo de precios elevándose en paralelo. El boxplot lo corrobora. El salto más notable se produce precisamente entre el rango más bajo y el resto, lo que sugiere que por debajo de un umbral de servicios (~10-11), el alojamiento queda relegado al segmento de precio más bajo independientemente de otras características. El `amenities_score` actúa, por tanto, como proxy de la calidad general del anfitrión: quienes equipan bien su piso tienden también a cuidar la ubicación, las fotografías y la atención al huésped.

El gráfico 4 es el más relevante para el modelo, pues cuantifica el impacto individual de cada servicio como diferencia de precio medio entre tenerlo y no tenerlo. El aire acondicionado lidera con una diferencia superior a los 55 €/noche, cifra que refleja la realidad climática de Málaga y convierte al climatizador en el servicio con mayor poder de fijación de precio del dataset. TV, cocina y lavadora ocupan el segundo nivel (elementos básicos). La terraza o balcón, pese a estar presente solo en el 40% de los anuncios, genera una diferencia de aproximadamente 22 €/noche, situándose como el único extra de calidad que compite en impacto con los servicios básicos.

Estos resultados sugieren que el `amenities_score` en su versión actual, que suma todos los servicios con igual peso, podría refinarse ponderando algunas amenities sobre otras. No obstante, dado que el modelo incluirá también las variables binarias individuales de cada servicio, la red neuronal tiene capacidad para aprender de forma implícita esta diferencia de pesos durante el entrenamiento, sin necesidad de definirla manualmente.

3.2.5. Correlaciones

La matriz de correlación confirma los hallazgos de los apartados anteriores y sirve como herramienta de control para identificar variables que deben ser excluidas del entrenamiento.



Las variables con mayor correlación positiva son todas de dimensión física: accommodates, bathrooms, bedrooms y beds. Este bloque confirma de forma cuantitativa que el tamaño del alojamiento es el principal predictor lineal del precio. En el rango moderado aparecen variables de calidad como amenities_score, has_balcony_or_terrace y review_scores_rating, coherentes con los análisis previos. En el extremo negativo, variables de actividad como reviews_per_month (-0.128) y estimated_occupancy_l365d (-0.127) correlacionan inversamente con el precio, reflejo del patrón ya observado: Los alojamientos más baratos acumulan más reservas y reseñas, mientras que los de precio alto operan con menor frecuencia de ocupación.

Variables como `distancia_playa_km` o `latitude` muestran correlaciones cercanas a cero. Esto no indica que carezcan de relevancia predictiva, sino que su relación con el precio es no lineal, como quedó demostrado en el análisis geográfico del apartado 3.2.3. Una regresión lineal clásica sería incapaz de capturar estas curvaturas, lo que refuerza técnicamente la elección de una red neuronal profunda como arquitectura del modelo.

Prevención de Data Leakage

La matriz de correlación revela dos variables que, pese a mostrar correlaciones muy elevadas con el precio, deben ser excluidas del conjunto de features de entrenamiento por constituir un riesgo de fuga de datos (data leakage).

La primera es `price_per_person` ($r = 0.659$), la variable con mayor correlación de todo el dataset. Su alta correlación es lógica, se trata de una variable construida dividiendo el propio precio entre la capacidad del alojamiento, por lo que incluirla en el modelo equivaldría a proporcionar la respuesta como input. La red neuronal aprendería a invertir matemáticamente esta operación en lugar de extraer patrones reales del mercado, generando métricas de validación engañosamente buenas que no se replicarían en producción.

La segunda es `estimated_revenue_l365d` y `estimated_occupancy_l365d`. Esta variable (revenue) es un target alternativo del propio modelo, calculado a partir del precio multiplicado por la ocupación estimada. Incluirla como feature de entrada permitiría a la red deducir el precio directamente desde los ingresos, algo que de cara a producción no sirve de nada, ya que un apartamento nuevo no contará con este dato. Ambas variables han sido eliminadas del dataset de entrenamiento final, quedando únicamente como targets de predicción.

3.3. El Cerebro Tabular: Red Neuronal Densa (DNN)

3.3.1 — Pipeline de Preprocesamiento

Antes de construir la red neuronal, fue necesario preparar un pipeline de preprocesamiento robusto que garantizase la calidad de los datos de entrada y la reproducibilidad del proceso tanto en entrenamiento como en producción.

Eliminación de Data Leakage

Como se adelantó en el apartado 3.2.5, el primer paso del pipeline consistió en purgar las variables que contenían el precio de forma implícita. La variable *price_per_person* se calcula dividiendo el precio entre la capacidad del alojamiento, por lo que incluirla equivaldría a proporcionar la respuesta como *input*: la red aprendería a invertir una operación matemática trivial en lugar de extraer patrones reales del mercado. Del mismo modo, *estimated_revenue_l365d* es el producto del precio por la ocupación estimada, convirtiéndola en un *target* alternativo enmascarado. Ambas variables, junto con *id* y *estimated_occupancy_l365d*, garantizando que el modelo nunca tenga acceso directo ni indirecto al valor que debe predecir.

Ingeniería de Categorías: Agrupación de *property_type*

El diagnóstico de columnas reveló que *property_type* presentaba 39 categorías únicas. Aplicar One-Hot Encoding directamente habría generado 38 columnas binarias nuevas, lo que añadiría ruido al entrenamiento sin aportar capacidad predictiva real. Por ello, se conservaron los 10 tipos de propiedad más frecuentes (agrupando el resto en *other*) encabezados por *Entire rental unit* con 4.129 registros. De este modo se retiene el 95% de la información relevante eliminando el ruido de la larga cola de tipos minoritarios.

ColumnTransformer: El Director de Orquesta

El preprocesado de variables mixtas (numéricas y categóricas) se centralizó en un ColumnTransformer de Scikit-Learn que aplica transformaciones diferentes a cada grupo de columnas dentro de un único objeto serializable. Su estructura se compone de dos ramas en paralelo:

- **Rama numérica** : los valores nulos se imputan con la mediana de cada columna. A continuación, el StandardScaler normaliza cada variable para que tenga media 0 y desviación típica 1. Imprescindible para el descenso de gradiente: sin él, variables con rangos muy dispares tendrían pesos iniciales desproporcionados, ralentizando o desestabilizando la convergencia.
- **Rama categórica** (Pipeline → SimpleImputer + OneHotEncoder): los valores nulos se imputan con la categoría más frecuente. El OneHotEncoder convierte cada categoría en columnas binarias independientes, evitando las jerarquías numéricas artificiales que introduciría un codificado ordinal. El parámetro *handle_unknown='ignore'* garantiza que, ante una categoría desconocida en producción —por ejemplo, un barrio nuevo no visto durante el entrenamiento—, el encoder asigne un vector de ceros en lugar de lanzar un error, haciendo el pipeline robusto para su despliegue en la API.

El parámetro `remainder='drop'` actúa como red de seguridad, eliminando cualquier columna que no haya sido asignada explícitamente a ninguna de las dos ramas.

División del Dataset: Train / Validación / Test

El dataset se dividió en tres conjuntos mutuamente excluyentes utilizando `train_test_split` con `random_state=42` para garantizar la reproducibilidad:

- **Train (64%):** El modelo ve estos datos y ajusta sus pesos con ellos.
- **Validación (16%):** El modelo no aprende de ellos; se usan durante el entrenamiento para monitorizar la generalización y como criterio del Early Stopping.
- **Test (20%):** Reservados en su totalidad hasta la evaluación final, simulan apartamentos completamente nuevos que llegarían a la API en producción.

La regla de oro del preprocesado se aplicó estrictamente: el `ColumnTransformer` realizó el `.fit_transform()` únicamente sobre el conjunto de Train, aprendiendo los parámetros de escalado (medias, desviaciones) y las categorías válidas solo a partir de esos datos. Para Validación y Test se aplicó exclusivamente `.transform()`, evitando así el *data leakage* sutil que se produciría si los parámetros de normalización se recalculasen incluyendo datos futuros. El preprocessor resultante fue serializado a disco mediante `joblib.dump()` para garantizar que la API de producción aplique exactamente la misma transformación que el modelo vio durante el entrenamiento.

El resultado final del pipeline fueron **35 variables de entrada** al modelo, distribuidas en los tres conjuntos con la forma (3.672, 35), (919, 35) y (1.148, 35) respectivamente.

3.3.2 — Arquitectura MLP

El modelo tabular de SmartHost Analytics se implementa como un Perceptrón Multicapa (*Multi-Layer Perceptron*, MLP), la arquitectura de red neuronal densa por excelencia para datos tabulares estructurados. En un MLP, cada neurona de una capa está conectada con todas las neuronas de la capa siguiente (*fully connected*), permitiendo al modelo aprender combinaciones arbitrariamente complejas de las 73 variables de entrada.

Estructura de capas

La arquitectura sigue un esquema de embudo decreciente en cuatro bloques:

Entrada (73) → Capa 1 (128) → Capa 2 (64) → Capa 3 (32) → Salida (1)

Cada uno de los tres bloques ocultos comparte la misma estructura interna: una capa lineal densa, seguida de normalización por lotes, función de activación y regularización por abandono. Esta simetría facilita el mantenimiento del código y asegura que cada capa aplique el mismo tratamiento a sus representaciones internas. La reducción progresiva de dimensionalidad —128, 64, 32 neuronas— obliga a la red a comprimir la información en representaciones cada vez más abstractas y generales, extrayendo los patrones más relevantes del mercado turístico malagueño.

Justificación de cada componente

- **nn.Linear**: la capa densa fundamental. Es el único componente que aprende; el resto son técnicas de estabilización y regularización.
- **BatchNorm1d**: normaliza las activaciones de cada mini-batch para que tengan media 0 y varianza 1 antes de pasar a la siguiente capa. Sin ésta, las distribuciones de activación se desplazan durante el entrenamiento (*internal covariate shift*), obligando a usar tasas de aprendizaje muy bajas y ralentizando la convergencia. Con BatchNorm1d, el modelo tolera tasas de aprendizaje más altas y converge en menos épocas.
- **ReLU** (*Rectified Linear Unit*): función de activación. Introduce la no-linealidad imprescindible para que la red pueda aprender relaciones complejas entre variables. Frente a alternativas clásicas como la sigmoide o la tangente hiperbólica, ReLU evita el problema del *vanishing gradient*: al no saturar para valores positivos, los gradientes fluyen sin atenuarse hacia las capas más profundas durante la retropropagación.

- **Dropout($p=0.2$):** durante cada *forward pass* del entrenamiento, apaga aleatoriamente el 20% de las neuronas de la capa. Al no poder depender siempre de las mismas conexiones, la red se ve forzada a aprender representaciones distribuidas y redundantes, lo que equivale a entrenar simultáneamente un ensamblado implícito de subredes. El resultado es un modelo significativamente más robusto frente al sobreajuste (*overfitting*).

Capa de salida: ausencia deliberada de activación

La capa de salida contiene una única neurona lineal sin función de activación. Esta resuelve una tarea de **regresión**, en la que el objetivo es predecir un valor continuo —el precio por noche en euros— que puede tomar cualquier valor positivo dentro de un rango amplio. Añadir una activación sigmoide o softmax limitaría el rango de predicción, haciendo imposible predecir precios reales. La salida lineal ilimitada es, por tanto, el único diseño correcto para este tipo de problema.

Capacidad del modelo

Con 20.289 parámetros entrenables —pesos y sesgos de todas las capas lineales—, la red tiene suficiente capacidad para capturar las interacciones no lineales entre las variables del mercado sin incurrir en el riesgo de memorización.

3.3.3 — Configuración RRNN para entrenamiento

Aceleración por GPU

El modelo se transfirió íntegramente a la GPU mediante, ejecutándose sobre una **NVIDIA GeForce RTX 4070 Laptop GPU** con backend CUDA. Frente a la ejecución en CPU, esto reduce el tiempo por época considerablemente, haciendo viable la experimentación iterativa con múltiples versiones del modelo.

Función de Pérdida: MSELoss

Como criterio de optimización se eligió el **Error Cuadrático Medio** (MSELoss).

La propiedad fundamental del MSE es que eleva al cuadrado cada error individual, penalizando de forma desproporcionada las predicciones muy alejadas del valor real. Un fallo de 50€ genera una pérdida 25 veces mayor que un fallo de 10€, lo que obliga a la red a priorizar la eliminación de los errores graves sobre el ajuste fino de los casos fáciles. Esta característica la hace especialmente adecuada para la predicción de precios de alojamiento, donde las desviaciones extremas —predecir 80€ para un apartamento de 250€— resultan inaceptables para el inversor.

No obstante, el MSE no es intuitivo como métrica de comunicación. Por ello, durante el entrenamiento se calculó en paralelo el **Error Absoluto Medio** (MAE) sobre el conjunto de validación, que sí expresa el error medio en euros reales y es la métrica que se reporta en los resultados finales.

Optimizador: Adam

La actualización de los 20.289 parámetros entrenables se realizó mediante el algoritmo **Adam** (*Adaptive Moment Estimation*) con una tasa de aprendizaje de 0.001. Manteniendo un historial de los gradientes recientes para cada parámetro individualmente, ajustando su tasa de actualización de forma adaptativa: los parámetros que reciben gradientes consistentes se actualizan con pasos más amplios, mientras que los parámetros con gradientes ruidosos se actualizan con cautela. Esta propiedad lo convierte en el optimizador de referencia en Deep Learning, ofreciendo convergencia rápida y robusta sin necesidad de ajuste manual del *learning rate*.

DataLoader y tamaño de batch

Los datos se sirvieron al modelo en mini-batches de 64 apartamentos mediante el DataLoader de PyTorch. Entrenar con el dataset completo de golpe (*full-batch*) saturaría la memoria de la GPU y produciría un gradiente demasiado promediado. El

parámetro `shuffle=True` en el loader de entrenamiento garantiza que el orden de los apartamentos sea aleatorio en cada época, evitando que la red memorice patrones espúreos debidos a la secuencia de los datos.

Early Stopping

Para detener el entrenamiento en el punto óptimo de generalización se implementó una estrategia de **Early Stopping**. En cada época, si la pérdida sobre el conjunto de validación (`val_loss`) mejora respecto al mínimo histórico, se reinicia el contador y se guarda una copia de los pesos actuales del modelo.

3.3.4 — Experimentación durante el entrenamiento y resultado final

El desarrollo del cerebro tabular no fue lineal. La versión final es el resultado de un proceso iterativo de hipótesis, prueba y análisis de resultados en el que cada versión diagnosticó el problema residual de la anterior. La siguiente tabla resume el recorrido:

Versión	Descripción	MAE val	R ² val
V1 — Baseline	Dataset con cap 600€, precio bruto	~30-32 €/noche	0.25-0.27
V2 — Transformación logarítmica	Mismo dataset, target $\log_{10}(\text{precio})$	28 €/noche	0.27-0.29
V3 — Price Cap	Cap 450€, precio bruto, sin log	24.89 €/noche	0.55-0.59

V1 — Baseline

El modelo de partida se entrenó sobre el dataset sin ningún tratamiento adicional del target, más allá de lo realizado en la fase 1 de limpieza. Los resultados —MAE de 30-32€ y R² en torno a 0.3— evidenciaron el problema: aunque el dataset ya había sido filtrado respecto al original, los apartamentos seguían generando gradientes desproporcionados durante el entrenamiento. La función MSELoss eleva al cuadrado cada error, por lo que predecir 200€ para un apartamento de 1000€ produce una penalización 28 veces mayor que cometer el mismo error absoluto en un apartamento de 100€. La red dedicaba una fracción excesiva de su capacidad de aprendizaje a estos casos extremos, en detrimento del segmento central del mercado donde se concentra el 95% de los apartamentos reales.

Esta hipótesis se intentó solucionar en la versión V2 mediante la transformación logarítmica.

V2 — Transformación Logarítmica

Para paliar la asimetría del target sin eliminar datos, se aplicó la transformación $\log_{10}(\text{precio})$ antes de entrenar, deshaciendo la transformación con $\exp(10 \cdot \text{pred})$ en la fase de evaluación para obtener métricas en euros reales, con esta transformación se pretendía que un error de 5€ en un apartamento barato pese lo mismo que uno de 100€ en un apartamento caro. Sin embargo, la mejora fue leve: el MAE cayó a 28€ y el r2 apenas fue 0.02 superior al anterior. Además, para un producto dirigido a inversores, esta simetría artificial no es deseable.

Por ello se llega a la hipótesis de probar a eliminar los pocos outliers restantes del dataset

V3 — Price Cap a 450€ (Modelo Ganador)

La solución definitiva fue quirúrgica: reducir el umbral de precio máximo a **450€**, equivalente al percentil 99 del dataset. Este ajuste eliminó únicamente el 1% de registros correspondientes a villas y áticos de ultra-lujo con escasa representatividad en el mercado turístico medio de Málaga y poca rotación. Con el foco de aprendizaje concentrado en el mercado real. El resultado fue el mejor de las tres versiones en todas las métricas: MAE de **25€ aprox** y R^2 de **0.55-0.59**, sin necesidad de transformar el target ni de añadir complejidad al pipeline. El modelo V3 se guarda en disco como `airbnb_mlp_PriceCAP.pt` junto con el `preprocessor_V3.pkl`, constituyendo el artefacto final del cerebro tabular.

3.4. Preprocesamiento de imágenes mediante script

El Problema de la Heterogeneidad Visual

Las redes neuronales convolucionales (CNN) operan mediante tensores matemáticos estrictos que exigen que todas las matrices de entrada posean dimensiones idénticas. Sin embargo, las fotografías extraídas mediante *web scraping* de los servidores de Airbnb presentan una casuística caótica en el mundo real: resoluciones enormemente variables (desde cámaras de baja calidad hasta formatos 4K), proporciones dispares (encuadres verticales, horizontales o panorámicos) y, en ocasiones, archivos parcialmente corruptos debido a microcortes en la descarga. Para solventar este problema de dimensionalidad sin destruir la información espacial del inmueble, se desarrolló un script de preprocesamiento "One-Shot" en Python.

La Solución Técnica: Escalar y Recorte Central

Se descartó el uso de un redimensionamiento directo ("aplastamiento"), ya que forzar una imagen rectangular a un formato cuadrado deforma el mobiliario, altera la percepción de profundidad y entrena a la red con perspectivas irreales. En su lugar, el script implementa una transformación en dos fases para alcanzar el estándar de entrada (224 x 224 píxeles) exigido por arquitecturas como ResNet34 o MobileNetV2:

1. **Escalado Proporcional (Resize 256):** El algoritmo identifica el lado más corto de la fotografía y lo reduce exactamente a 256 píxeles. El lado opuesto se ajusta automáticamente manteniendo la proporción (relación de aspecto o *aspect ratio*) original, evitando cualquier tipo de deformación visual.
2. **Recorte Central (CenterCrop 224):** Sobre la nueva imagen intermedia, se extrae un cuadrado perfecto de 224 x 224 píxeles originado en el centro geográfico del lienzo. Esta decisión arquitectónica se alinea con las normas de composición fotográfica en el sector inmobiliario (*Real Estate*), donde el punto focal de mayor interés (la cama, el sofá o la ventana principal) se sitúa invariablemente en el centro del encuadre, desechando únicamente los márgenes irrelevantes.

Características Clave y Robustez del Sistema (Fault Tolerance)

El desarrollo de este script priorizó la tolerancia a fallos frente a un volumen masivo de datos (~5.700 archivos). Sus características más destacadas incluyen:

- **Inmunidad ante anomalías de archivo:** Mediante la manipulación a bajo nivel de la librería PIL (`ImageFile.LOAD_TRUNCATED_IMAGES = True`), el sistema es capaz de procesar imágenes que quedaron truncadas durante el *scraping*. Además, se suprimió el límite de píxeles (`Image.MAX_IMAGE_PIXELS = None`) para prevenir el colapso del programa ante fotografías con resoluciones anómalas (superiores a 89 megapíxeles) que de otro modo generarían un error por prevención de bombas de descompresión.
- **Trazabilidad y Sincronización:** El script actúa como una compuerta de validación de calidad. Emite un informe final que contabiliza imágenes procesadas, omitidas y errores irreversibles. La identificación de estos errores es crítica para garantizar una alineación perfecta en la fase de Fusión Multimodal.

3.5. El Cerebro Visual : Red Neuronal Convolutacional (CNN)

3.5.1. Sincronización y Preparación del Dataset Visual

Alineación de datos e integridad del conjunto

Antes de iniciar cualquier entrenamiento visual, el proyecto afronta un desafío crítico de ingeniería de datos: garantizar que cada apartamento disponga tanto de sus datos tabulares como de su fotografía de portada. El proceso de sincronización cruza los identificadores únicos del dataset tabular filtrado (listingV5PriceCapID.csv, con el price cap aplicado) con los archivos físicos de imagen almacenados en FrontImages224/. Para cada fila del CSV se construye la ruta esperada <id>.jpg y se verifica su existencia en disco. Solo los apartamentos con "pareja" completa —datos estructurados y fotografía— sobreviven al filtro, resultando en **5.667 registros sincronizados**. Esta restricción es indispensable para garantizar la integridad de la futura fusión multimodal: si un apartamento careciera de imagen, no podría ser evaluado por el sistema completo.

El split entre entrenamiento y validación se aplica con random_state=42 de forma idéntica en los tres notebooks, garantizando que las particiones train/val son **estrictamente consistentes** entre todos los modelos. Esto es un requisito imprescindible: los embeddings extraídos por la CNN deben corresponder exactamente a los mismos apartamentos que los features tabulares del modelo XGBoost, o la fusión multimodal produciría resultados incoherentes.

Pipeline de transformación de imagen

Cada imagen atraviesa un pipeline de preprocesamiento diferenciado según la fase. Las fotografías ya fueron redimensionadas a 224×224 píxeles en un paso de preparación previo. En el momento del entrenamiento se aplican dos pipelines:

Pipeline de entrenamiento (*data augmentation* enriquecido, implementado en los tres modelos):

- RandomHorizontalFlip(p=0.5) — volteo horizontal aleatorio. Evita que el modelo aprenda correlaciones espurias entre la disposición espacial de los muebles y el precio.
- ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1) — variaciones de iluminación y color para simular condiciones fotográficas diversas.
- RandomRotation(degrees=15) + RandomAffine(translate=(0.05, 0.05)) — tolerancia a variaciones de composición.
- Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) — normalización con los estadísticos de ImageNet, imprescindible para que los pesos preentrenados operen en su rango esperado.

Pipeline de validación (determinista): únicamente ToTensor() + Normalize(), garantizando que las métricas de validación sean reproducibles y comparables entre épocas y modelos.

Los precios se normalizan mediante Z-score ($\text{PRICE_MEAN} \approx 111.77 \text{ €}$, $\text{PRICE_STD} \approx 61.67 \text{ €}$), estabilizando el gradiente durante el entrenamiento.

3.5.2. Arquitectura: Proceso de Selección y Transfer Learning

Justificación del Transfer Learning

Entrenar una red convolucional profunda desde cero con ~5.667 imágenes resulta inviable: las redes modernas tienen decenas de millones de parámetros y requieren millones de ejemplos para generalizar sin sobreajuste. Se optó por **Transfer Learning**: utilizar arquitecturas preentrenadas en ImageNet (1,2 millones de imágenes, 1.000 clases) que ya "saben" reconocer bordes, texturas, materiales y objetos cotidianos. Para interiores de apartamentos, este conocimiento es directamente aplicable: distinguir si un suelo es de parquet o baldosa, si los electrodomésticos son modernos, o si la iluminación es cálida, son exactamente el tipo de características que ImageNet ha enseñado a estas redes.

Los tres modelos comparten la misma estrategia general de dos fases: un **warm-up** inicial con el backbone congelado, seguido de un **fine-tuning** donde se desbloquean las capas más profundas y específicas de dominio.

MobileNetV2 es la arquitectura más ligera (3,4 M parámetros, convoluciones separables en profundidad). Su cabeza original classifier —Dropout(0.2) → Linear(1280→1000)— se sustituye por Dropout(0.2) → Linear(1280→1). Durante el warm-up se entrena solo la cabeza; en el fine-tuning se desbloquean features[14:] (los 4 últimos bloques + el conv expensor de 1280 canales), usando un LR diferenciado más bajo para el backbone ($1e-5$) que para la cabeza ($1e-4$). El modelo convergió rápidamente —solo 30 épocas hasta el Early Stopping— pero su menor capacidad representacional se refleja en un R^2 de apenas 0,109.

EfficientNet-B0, con su arquitectura MBConv con Squeeze-and-Excitation y 237 capas efectivas, es la que mayor rendimiento ImageNet ofrece por parámetro (77,1% con solo 5,3 M). Su profundidad, sin embargo, introdujo mayor variabilidad en la curva de entrenamiento con este dataset reducido.

ResNet34 ofreció el mejor equilibrio entre capacidad representacional y estabilidad de entrenamiento, logrando el mejor R^2 (0,174) y el menor Val MAE (36,82 €) de los tres modelos. Sus bloques residuales con *skip connections* facilitan el flujo de gradiente y previenen el gradiente desvaneciente incluso en redes de 34 capas.

Modificación de la arquitectura de ResNet34 para regresión

La capa fc original de ResNet34 —diseñada para clasificar en 1.000 categorías de ImageNet— se reemplaza íntegramente por una **cabeza de regresión** de tres capas densas con regularización progresiva:

text

```
avgpool (512-dim) → Linear(512→256) → ReLU → Dropout(0.5)
→ Linear(256→128) → ReLU → Dropout(0.3) → Linear(128→1)
```

El Dropout decreciente (0,5 → 0,3) aplica mayor regularización en las capas iniciales, donde la red aprende representaciones más generales, y relaja la restricción conforme se aproxima a la predicción final. Al inicio del entrenamiento, solo los **164.353 parámetros** de esta cabeza son entrenables, frente a los 21.284.672 del backbone congelado.

3.5.3. El Proceso de Entrenamiento Visual (ResNet34)

Estrategia de congelación en dos fases

El entrenamiento se estructura en dos fases bien diferenciadas para proteger el conocimiento visual previo:

Fase 1 — Warm-up (épocas 1–10): Todo el backbone de ResNet34 permanece congelado. Solo los 164.353 parámetros de la cabeza fc son entrenables. Los resultados de esta fase muestran una convergencia progresiva y estable: el Val MAE baja de 41,45 € (época 1) hasta 38,77 € (época 10), con un R^2 que pasa de -0,045 a **+0,100**, indicando que la cabeza aprende a interpretar las features visuales de ImageNet de forma útil para el dominio de precios.

Fase 2 — Fine-tuning (época 11 en adelante): Se desbloquean layer3 y layer4 —las dos últimas capas residuales, las más específicas de dominio—, manteniendo layer1 y layer2 congeladas. El learning rate se reduce a $LR_FINETUNE = 5e-5$ para ajustes finos sin destruir la especialización de ImageNet. El efecto es inmediato y significativo: el Train MAE cae rápidamente de ~38 € hasta 24 €, y el Val MAE alcanza su mínimo de **36,82 €** en la época 24 ($R^2 = 0,174$).

Hardware, función de pérdida y optimizador

El entrenamiento se ejecutó sobre una **GPU NVIDIA GeForce RTX 4070 Laptop** (8,6 GB VRAM), con un tiempo total de ~65 minutos para 39 épocas hasta activar el Early Stopping.

- **Función de pérdida:** L1Loss (MAE), más robusta ante los *outliers* de precio que MSE, penalizando linealmente los errores.
- **Optimizador:** Adam con $weight_decay=1e-4$.
- **Scheduler:** ReduceLROnPlateau (factor 0,5, patience 5), que reduce el LR automáticamente: $5e-5 \rightarrow 2,5e-5 \rightarrow 1,25e-5 \rightarrow 6,25e-6$ a lo largo del fine-tuning.

- **Early Stopping** con patience=15: el entrenamiento se detuvo en la **época 39** al no mejorar el Val MAE desde la época 24, restaurando automáticamente los pesos del mejor checkpoint guardado en models/resnet34_best.pt.

3.5.4. Análisis de Resultados e Insights Visuales

Rendimiento comparado de los tres modelos

Modelo	Parámetros	Top-1 ImageNet	Mejor Val MAE	Mejor Val RMSE	Mejor Val R ²	Época óptima	Tiempo
MobileNetV2	3,4 M	71,8%	37,51 €	59,34 €	0,109 (10,9%)	15	~30 épocas/30 min aprox
EfficientNet-B0	5,3 M	77,1%	39, 14€	59,75€	0.114	32	45 min
ResNet34	21,8 M	73,3%	36,82 €	57,13 €	0,174 (17,4%)	24	65 min / 39 ep

Los tres modelos presentan un R² sensiblemente inferior al del modelo tabular, confirmando que la imagen sola no puede tasar con precisión. MobileNetV2, a pesar de su mayor eficiencia computacional (converge en menos épocas), no alcanza la capacidad de representación de ResNet34. La diferencia entre Train MAE (24,71 €) y Val MAE (36,82 €) en ResNet34 revela cierto sobreajuste, típico con datasets visuales reducidos en dominios de alta variabilidad estética.

Conclusión de negocio: la imagen como modulador estético

El análisis comparado lleva a una conclusión de alto valor estratégico para el proyecto: **la imagen no dicta el precio base del apartamento, sino que actúa como una variable más**. La visión artificial por sí sola no puede tasar una propiedad con precisión absoluta, ya que carece de contexto geográfico, dimensional y estructural —no sabe cuántas habitaciones tiene el piso, en qué barrio está ubicado ni cuál es su superficie en m²—. Sin embargo, es capaz de captar con notable sensibilidad matices de calidad y modernidad que los datos numéricos ignoran por completo: si los acabados son de lujo, si la decoración está actualizada, si la iluminación transmite calidez, o si el espacio parece amplio y bien cuidado. Esta capacidad complementaria es precisamente la razón de ser de la arquitectura de fusión multimodal: combinar el rigor geoestructural de los datos tabulares con la inteligencia estética de la visión artificial.

3.5.5. Extracción de Embeddings para la Fusión Multimodal

Generación de vectores de características

El resultado final de la fase de Computer Vision no es únicamente la predicción de precio de la CNN, sino la extracción de un **vector de características de 512 dimensiones** —los *embeddings* visuales— generado por la capa avgpool de ResNet34, justo antes de la cabeza de regresión. Este vector encapsula el "resumen visual" completo de cada apartamento: la red ha destilado toda la información de los ~150.000 píxeles de la imagen

en 512 números que capturan texturas, materiales, luminosidad y composición. Cualitativamente, estos 512 valores son mucho más ricos e informativos que el único escalar de precio predicho, porque el modelo de fusión puede explotarlos de formas no lineales y complementarlas con las features tabulares.

Exportación a archivos .npy

Los embeddings de todos los apartamentos se extraen en modo `model.eval()` —sin gradiente ni dropout activo— pasando cada imagen por el backbone hasta `avgpool`, y se almacenan en archivos NumPy:

- `embeddings_train.npy` — matriz de forma (4.533, 512) con los embeddings del conjunto de entrenamiento.
- `embeddings_val.npy` — matriz de forma (1.134, 512) con los embeddings de validación.

Estos archivos constituyen el puente entre la fase de Computer Vision y la fase de **fusión multimodal**, donde serán concatenados con las características tabulares del modelo XGBoost para alimentar al modelo integrador final. El uso del mismo `random_state=42` en los tres notebooks garantiza que los índices train/val son estrictamente idénticos a los del modelo tabular, condición indispensable para que la fusión sea metodológicamente correcta.

4. ARQUITECTURA DEL MODELO DUAL Y JUSTIFICACIÓN

4.1. Rama A: El Cerebro Visual CNN (detallado en 3.5)

La red ResNet34 preentrenada con los pesos de **ImageNet-1k** se cargó y su backbone fue **congelado** durante las primeras fases del entrenamiento, permitiendo que únicamente la capa de regresión añadida aprendiera de los datos del proyecto.

4.1.2. Preprocesamiento de Imagen: Script redimensionado (Detallado en 3.4)

La resolución de entrada 224×224 píxeles no es arbitraria: es el estándar histórico de ImageNet derivado de los requerimientos computacionales y perceptuales de las CNNs profundas. La justificación matemática es la siguiente.

Una imagen de resolución original típica (por ejemplo, 1200×800 píxeles) sometida directamente a la red generaría un mapa de características de dimensión inaceptable. Dado que ResNet34 aplica un stride de 2 en sus primeras capas y cuatro bloques residuales que reducen la resolución espacial.

Este mapa es el que alimenta la capa de *average pooling* global, comprimiéndolo en un único vector de 512 dimensiones. Reducir la entrada a 112×112 limitaría la capacidad de detectar patrones espaciales medianos (ventanas, sofás, distribución de la habitación), mientras que una entrada de 448×448 cuadruplicaría el coste computacional sin mejora proporcional en el dominio de interiores.

Adicionalmente, durante la fase de entrenamiento se aplicó **Data Augmentation** mediante volteo horizontal aleatorio (RandomHorizontalFlip) para incrementar la variabilidad del conjunto de entrenamiento sin necesidad de nuevas imágenes.

Finalmente, los valores de cada canal RGB se normalizaron con la media y desviación estándar de ImageNet, garantizando que los gradientes durante el entrenamiento operen en la misma escala numérica con la que los pesos preentrenados fueron optimizados.

4.1.3. Extracción del Vector de Embeddings Visuales

La capa de clasificación final de ResNet34 (originalmente una capa `nn.Linear(512, 1000)` para las 1.000 clases de ImageNet) fue reemplazada por una capa `nn.Linear(512, 1)` para la tarea de regresión. Sin embargo, en la arquitectura multimodal final, esta cabeza fue suprimida para **extraer el vector de 512 dimensiones** que precede al clasificador original.

Este vector, resultado del *average pooling* global sobre el mapa de características final, actúa como el **"resumen estético" del apartamento**: una representación densa donde cada una de las 512 dimensiones codifica un rasgo visual aprendido (luminosidad global, presencia de texturas cálidas, densidad de mobiliario, etc.). Este vector es la contribución de la Rama A al modelo de fusión.

4.2. Rama B: El Cerebro Tabular DNN (detallado en 3.3)

4.2.1. Estructura de la Red Densa

La Rama B es una red neuronal densa (*Multi-Layer Perceptron*, MLP) que procesa las variables estructurales del alojamiento. La arquitectura elegida consta de tres capas ocultas con la siguiente progresión de neuronas: $256 \rightarrow 128 \rightarrow 64$, seguidas de una capa de salida de 1 neurona.

text

Entrada (30 features) $\rightarrow$ [256, BN, ReLU, Dropout(0.3)]
 $\rightarrow$ [128, BN, ReLU, Dropout(0.2)]
 $\rightarrow$ [64, BN, ReLU]
 $\rightarrow$ Embedding tabular (64-dim)

Nota de implementación: La guía inicial del proyecto contemplaba una arquitectura más reducida de $128 \rightarrow 64 \rightarrow 32$ neuronas. Durante el desarrollo se amplió a $256 \rightarrow 128 \rightarrow 64$ tras comprobar que la arquitectura reducida no capturaba suficientes interacciones entre variables de ingeniería de características, obteniendo un Val MAE superior en $\approx 2\epsilon$.

4.3. Fusión Multimodal (*Late Fusion*)

4.3.1. Mecanismo de Concatenación

La arquitectura de fusión implementada corresponde al paradigma de **Late Fusion** (fusión tardía): cada rama procesa su modalidad de forma independiente hasta producir un vector de representación, y ambos vectores se concatenan en un único tensor antes de la capa de decisión final.

Nota de implementación: La guía inicial estimaba un supervector de 547 dimensiones (35 features tabulares + 512 visuales). La implementación final opera sobre **576 dimensiones** (64 del embedding tabular + 512 del embedding visual), dado que la Rama B genera un embedding de 64 dimensiones — no pasa directamente las 30 features de entrada — lo que enriquece la representación tabular antes de la fusión. El número exacto de features tabulares de entrada fue **30** tras el proceso de ingeniería de características y eliminación de *data leakage*.

Este diseño de Late Fusion frente a la alternativa de Early Fusion (concatenar las *features* brutas antes de cualquier procesamiento) tiene una ventaja técnica clave: cada rama puede ser entrenada y especializada en su propia modalidad antes de que la capa de fusión aprenda a combinarlas. Esto es especialmente relevante dado que los 512 embeddings visuales de ResNet34 son el resultado de millones de parámetros preentrenados cuya escala y semántica es incompatible con las 30 variables tabulares sin una transformación intermedia.

4.3.2. Cabeza de Regresión Final

El supervector de 576 dimensiones pasa por una **cabeza de regresión** de dos capas:

$[576] \rightarrow \text{Linear}(576, 64) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(64, 1)$

La neurona de salida tiene **activación lineal** (sin función de activación), lo que permite que la red produzca cualquier valor real positivo como estimación del precio por noche, en lugar de estar restringida al rango $[0,1]$ de una sigmoide o al rango positivo de una ReLU.

4.4. Justificación Matemática de la *Loss Function*

4.4.1. Regresión vs. Clasificación

A diferencia de los ejemplos canónicos de la literatura de redes neuronales (clasificación de imágenes, detección de spam), SmartHost Analytics aborda un **problema de regresión continua**: la variable objetivo price toma valores reales en el intervalo [18,450] €/noche, sin categorías discretas.

Esta función penaliza la divergencia entre distribuciones de probabilidad y **no tiene sentido** cuando el target es un valor continuo como el precio.

4.4.2. MSE como Loss de Entrenamiento

Para el entrenamiento se utilizó el **Error Cuadrático Medio** (*Mean Squared Error*, MSE):

La elección del MSE como función de pérdida durante la optimización se justifica por su propiedad de **penalización cuadrática**: un error de 40€ contribuye cuatro veces más a la pérdida que un error de 20€. Esto obliga al optimizador (Adam) a priorizar la eliminación de los errores más grandes, forzando al modelo a no desviarse de forma grave del precio de mercado real — condición especialmente crítica en un sistema de apoyo a la decisión de inversión.

4.4.3. MAE como Métrica de Evaluación

Dado que la distribución de precios presenta asimetría positiva (confirmada en el EDA: mediana 98€ vs. media 119€), el MSE amplificaría desproporcionadamente el impacto de los pocos apartamentos de precio alto durante la evaluación. Por ello, la **métrica principal de referencia** es el **Error Absoluto Medio** (*Mean Absolute Error*, MAE):

El MAE es robusto a *outliers* y tiene una interpretación económica directa: un MAE de 21.47€ significa que el modelo se equivoca, en promedio, 21.47€ por noche en su estimación del precio de mercado, independientemente de si es un estudio de 50€ o un ático de 400€.

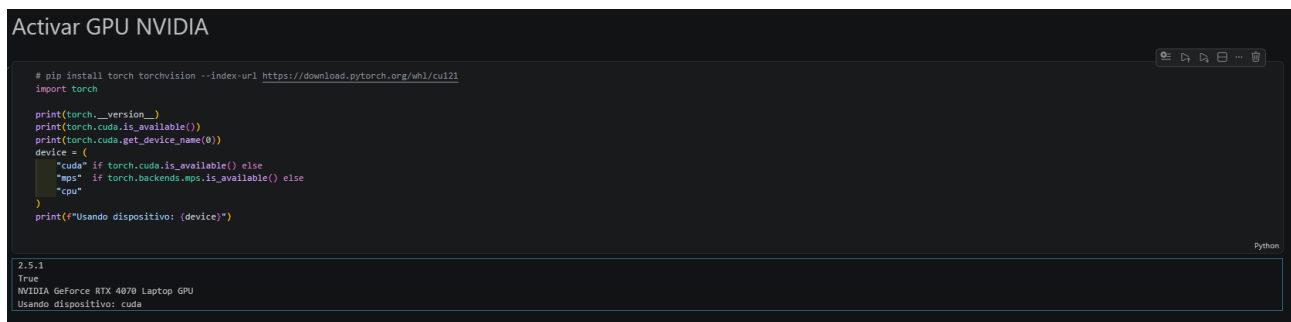
5. ENTRENAMIENTO Y OPTIMIZACIÓN

5.1. Aceleración por Hardware: GPU y Paralelización por Tensores

El entrenamiento de la red multimodal se ejecutó sobre una **NVIDIA GeForce RTX 4070 Laptop GPU** (8 GB VRAM, arquitectura Ada Lovelace), aprovechando la capacidad de cómputo paralelo masivo que ofrecen las GPU modernas frente a la ejecución secuencial en CPU.

La clave de esta aceleración reside en la naturaleza del cálculo neuronal: durante cada paso de entrenamiento, el modelo debe multiplicar matrices de miles de dimensiones para todos los elementos del *batch* de forma simultánea. Una CPU de uso general ejecuta estas operaciones de forma mayoritariamente secuencial, mientras que una GPU con miles de núcleos CUDA las paraleliza completamente. En términos prácticos, el entrenamiento de la CNN con ResNet34 (39 épocas) sobre 4.212 muestras de entrenamiento se completó en **30 minutos**, un tiempo que en CPU habría ascendido a horas.

La implementación sigue el patrón estándar de PyTorch para código agnóstico al dispositivo:



```
Activar GPU NVIDIA

# pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
import torch

print(torch.__version__)
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
device = (
    "cuda" if torch.cuda.is_available() else
    "mps" if torch.backends.mps.is_available() else
    "cpu"
)
print(f"Usando dispositivo: {device}")

2.5.1
True
NVIDIA GeForce RTX 4070 Laptop GPU
Usando dispositivo: cuda
```

Figura 5.1. Código activación GPU

Este diseño garantiza que el código sea **portátil**: en un entorno sin GPU (por ejemplo, el servidor de producción de la API o Google Colab en su versión gratuita), la línea `torch.cuda.is_available()` devuelve `False` y toda la ejecución migra automáticamente a CPU sin modificar una sola línea de código.

El entrenamiento se realizó con un **batch size de 64 muestras**: cada iteración del bucle interior procesa simultáneamente 64 apartamentos, cada uno con su vector tabular de 30 dimensiones y su embedding visual de 512 dimensiones. Este tamaño de *batch* representa un equilibrio empíricamente probado entre la estabilidad del gradiente (batches muy pequeños producen actualizaciones ruidosas) y el aprovechamiento de la memoria de la GPU (batches muy grandes saturan la VRAM).

5.2. Protocolo de Entrenamiento: Warm-up y Fine-tuning

El entrenamiento se estructuró en **dos fases secuenciales**, una decisión de diseño dictada por la naturaleza del Transfer Learning y la asimetría entre los parámetros preentrenados de ResNet34 y los pesos inicializados aleatoriamente de la cabeza de regresión.

Fase 1 — Warm-up con Backbone Congelado (épocas 1–10)

Durante las primeras épocas, los pesos del backbone de ResNet34 se mantienen **congelados** (`requires_grad = False`). Solo se entrenan los parámetros de la Rama Tabular y la cabeza de regresión final. La justificación técnica es crítica: si se descongelara el backbone desde el inicio, los gradientes de error provenientes de la cabeza de regresión no inicializada (con errores iniciales del orden de 100€) se propagarían hacia los pesos preentrenados de ResNet, destruyendo las representaciones visuales aprendidas sobre ImageNet en cuestión de pocas iteraciones. El warm-up actúa como estabilizador, permitiendo que la cabeza aprenda primero a producir predicciones razonables antes de que el backbone sea ajustado.

Fase 2 — Fine-tuning Parcial (a partir de la época 11)

Una vez estabilizada la loss de validación durante el warm-up, se **descongelan parcialmente** las capas más profundas de ResNet34 (`layer3` y `layer4`), que codifican características de alto nivel (semántica visual de los espacios). Las capas iniciales del backbone permanecen congeladas, ya que detectan bordes y texturas básicas universales que no requieren adaptación al dominio de interiores.

El learning rate de fine-tuning (5×10^{-5} a 55×10^{-5}) es deliberadamente un orden de magnitud inferior al del warm-up (5×10^{-4} a 45×10^{-4}) para ajustar los pesos preentrenados con pasos pequeños, preservando el conocimiento adquirido en ImageNet mientras se especializa en fotografías de apartamentos.

5.3. Optimizador: Adam con Learning Rate Adaptativo

Se utilizó el optimizador **Adam** (*Adaptive Moment Estimation*) con una tasa de aprendizaje inicial de $\alpha = 5 \times 10^{-4}$.

La ventaja de Adam sobre el Descenso de Gradiente Estocástico (SGD) clásico es que **adapta automáticamente el learning rate por parámetro**: los parámetros que reciben gradientes grandes (como las neuronas de la capa de entrada que corresponden a variables de alta varianza) se actualizan con pasos más pequeños, mientras que los parámetros con gradientes pequeños reciben pasos más grandes. Esto es especialmente beneficioso en arquitecturas multimodales donde la magnitud de los gradientes provenientes de la Rama Tabular y la Rama Visual puede diferir en órdenes de magnitud.

Adicionalmente, se configuró un **scheduler ReduceLROnPlateau** que monitoriza el Val MAE y reduce el learning rate a la mitad cuando éste no mejora durante 5 épocas consecutivas.

La evolución del learning rate durante el entrenamiento final muestra cuatro reducciones sucesivas hasta alcanzar 3.13×10^{-5} , momento en el que el Early Stopping detuvo el entrenamiento. Este patrón evidencia que el optimizador exploró el espacio de parámetros de forma sistemática y que la convergencia alcanzada fue genuina, no producto de un learning rate excesivamente bajo desde el inicio.

5.4. Técnicas Anti-Sobreaajuste

5.4.1. Dropout

El Dropout es una técnica de regularización que, durante cada iteración de entrenamiento, **desactiva aleatoriamente** un porcentaje de neuronas de la capa (fijando su salida a cero) con una probabilidad p . Las neuronas desactivadas no contribuyen al *forward pass* ni reciben actualización en el *backward pass*.

La justificación estadística es que el Dropout equivale aproximadamente a entrenar un **ensemble de $2n$ redes diferentes** (siendo n el número de neuronas), promediando sus predicciones en la fase de inferencia. Esto impide que la red desarrolle dependencias fuertes entre neuronas concretas — es decir, que memorice patrones específicos de los apartamentos del conjunto de entrenamiento en lugar de aprender reglas generales del mercado.

En la arquitectura implementada se aplicaron dos capas de Dropout con probabilidades diferenciadas:

Capa	Probabilidad de Dropout	Justificación
Tras la primera capa oculta (256→128)	$p = 0.3$	Mayor regularización en la capa más ancha
Tras la segunda capa oculta (128→64)	$p = 0.2$	Regularización moderada conforme se reduce la dimensionalidad

Nota sobre los experimentos de regularización: Durante el desarrollo se realizaron tres experimentos sistemáticos de hiperparámetros aumentando el Dropout hasta $p = 0.5/0.4$ y añadiendo Weight Decay de 10^{-3} a 10^{-1} , con el objetivo de reducir el gap de 7.99€ entre Train MAE y Val MAE. Los resultados demostraron que una regularización más agresiva reducía efectivamente el overfitting (gap ≈ 3.93 €) pero empeoraba el Val MAE absoluto hasta 22.56€, es decir, el modelo generaliza peor en datos nuevos. Esto es coherente con el volumen disponible de 4.212 muestras de entrenamiento: con pocos datos, la regularización fuerte introduce sesgo que supera al beneficio de la reducción de varianza. Se mantuvo la configuración original.

5.4.2. Early Stopping

El Early Stopping es una técnica de regularización implícita que detiene el entrenamiento cuando el **error de validación deja de mejorar** durante un número de épocas consecutivas definido como paciencia (*patience*). Se implementó una clase EarlyStopping personalizada con los siguientes parámetros:

- **Patience = 15**: el entrenamiento continúa hasta que el Val MAE no mejora en 15 épocas consecutivas. Este valor equilibra la tolerancia a fluctuaciones temporales del gradiente (patience muy bajo detendría prematuramente el entrenamiento ante mesetas transitorias) y la eficiencia computacional (patience muy alto entrenaría épocas innecesarias una vez el modelo ha convergido).
- **Delta = 0.01**: solo se considera mejora real una reducción del Val MAE de al menos 0.01€, filtrando oscilaciones numéricas insignificantes.
- **Guardado del mejor modelo**: en cada época que supera el umbral de mejora, la clase serializa los pesos del modelo en memoria (`best_weights = copy.deepcopy(model.state_dict())`). Al finalizar el entrenamiento, estos pesos son restaurados automáticamente, garantizando que el modelo desplegado corresponde al momento de máxima generalización y no al último paso de optimización.

El entrenamiento se detuvo en la **época 50**, restaurando los pesos de la **época 35** como mejor modelo, con un Val MAE de **21.47€** y un R^2 de **0.692**.

6. RESULTADOS, MÉTRICAS E INTERPRETABILIDAD

6.1. Análisis de Métricas Finales

6.1.1. El Salto de Precisión: Del Modelo Tabular al Multimodal

El objetivo central de SmartHost Analytics era demostrar empíricamente que la incorporación de la señal visual mejora la capacidad predictiva del sistema respecto al modelo tabular base. Los resultados confirman esta hipótesis de forma inequívoca:

Modelo	Val MAE	Val RMSE	R ²	Arquitectura
MLP Tabular (V3, Price Cap 450€)	~24.89€	~38.2€	0.55– 0.59	35 features → 128→64→32→1
CNN ResNet34 (solo visual)	36.82€	57.13€	0.174	Embeddings 512-dim → regresin
MultimodalMLP (fusión)	21.47€	35.45€	0.692	576-dim → 256→128→64→1

La red multimodal reduce el Val MAE en aproximadamente **3.4€ por noche** respecto al mejor modelo tabular, y mejora el R² en más de **0.10 puntos** (de 0.59 a 0.692). Esta mejora, aunque contenida en valor absoluto, tiene una interpretación estadística precisa: el modelo multimodal es capaz de explicar el **69.2% de la varianza del precio**, frente al 59% del modelo tabular. El 10.2% adicional de varianza explicada es directamente atribuible a la información visual — es la cuantificación matemática de lo que el mercado paga por la estética.

La CNN en solitario, con un R² de solo 0.174, confirma la hipótesis cualitativa establecida en el EDA: la imagen no dicta el precio base de un apartamento, carece de contexto geoestructural (no conoce el barrio, la superficie ni el número de habitaciones). Su valor no es predictivo de forma aislada, sino **modulador**: enriquece y matiza la estimación tabular con información que los números no pueden capturar.

6.1.2. Significado Económico del MAE

Un Val MAE de **21.47€/noche** sobre un mercado con precio medio de 112€/noche representa un error relativo del ~19%. Para contextualizar este resultado con precisión, es necesario considerar el **ruido irreducible** inherente al mercado de alquiler vacacional:

Dos apartamentos con características físicas y visuales idénticas, en el mismo edificio, pueden tener precios de 90€ y 130€ por noche dependiendo de la estrategia del anfitrión, el historial de reseñas acumulado, la temporada no observada en el dataset, y la competencia local en ese momento exacto de scraping.

Este ruido no es modelable con ninguna arquitectura, por abundantes que sean los datos. Representa el margen de **subjetividad del mercado**, y un MAE de 21€ sobre un precio medio de 112€ implica que el sistema ya ha aprendido a aproximarse al techo de lo que los datos disponibles permiten predecir.

El impacto económico para el inversor es concreto. Comparando el modelo tabular con el multimodal:

Reducción de error: $24.89\text{€} \rightarrow 21.47\text{€} = 3.42\text{€}$ por noche

Ocupación estimada: ~180 noches/año (tasa media de mercado en Málaga)

Mejora en precisión anual: $3.42\text{€} \times 180 = 615.60\text{€/año}$ por apartamento

Para un inversor que gestiona una cartera de 5 apartamentos, la diferencia entre tasar con el modelo tabular y tasar con el modelo multimodal equivale a **más de 3.000€ anuales en precisión de facturación**: precios mejor calibrados al mercado reducen las noches vacías por sobreprecio y los ingresos perdidos por infravaloración.

6.2. Análisis del Veredicto: Overfitting y Elección de Configuración

6.2.1. El Gap Train/Val en Perspectiva

El modelo final presenta un gap entre Train MAE (13.48€) y Val MAE (21.47€) de **7.99€**. Durante el desarrollo se realizaron tres experimentos sistemáticos para reducir este gap mediante regularización más agresiva:

Config	Val MAE	Train MAE	Gap	R ²
Original (Dropout 0.3/0.2)	21.47€	13.48€	7.99€	0.692
Dropout 0.5/0.4 + WD 1e-3	22.56€	18.63€	3.93€	0.672
Arquitectura pequeña + WD 1e-3	22.73€	17.81€	4.92€	0.683
Arquitectura pequeña + WD 1e-4	22.81€	19.73€	3.08€	0.659

Los experimentos demuestran un resultado contraintuitivo pero estadísticamente coherente: **reducir el gap no mejora el rendimiento real**. Las configuraciones con menor diferencia train/val producen sistemáticamente peores Val MAE y peores R². Esto responde a la ecuación fundamental del aprendizaje automático, el compromiso sesgo-varianza (*bias-variance tradeoff*):

Con solo 4.212 muestras de entrenamiento, la regularización agresiva reduce la varianza (el overfitting) pero introduce sesgo suficiente como para que el error total en validación aumente. El gap de 7.99€ en la configuración original es, por tanto, **aceptable y esperado** dado el tamaño del dataset: no es una señal de que el modelo sea deficiente, sino de que está operando al límite de lo que los datos disponibles permiten.

6.2.2. Justificación de la Elección Final

La métrica que determina el comportamiento del modelo con usuarios reales —apartamentos no vistos durante el entrenamiento— es el **Val MAE**, no el gap train/val. La configuración original, con Val MAE = 21.47€ y R² = 0.692, es la que mejor responde a la pregunta crítica del proyecto: *¿cuánto se equivoca el sistema cuando un inversor introduce un apartamento nuevo?*

Las configuraciones con menor gap responden mejor a la pregunta académica de *¿qué tan uniforme es el aprendizaje?*, pero en un sistema de apoyo a la decisión de inversión,

la respuesta correcta siempre prioriza el rendimiento en datos no vistos sobre la estética de las curvas de aprendizaje. La configuración original fue mantenida.

6.3. Análisis de Residuos

El análisis de residuos, calculado sobre el conjunto de test (**20% del dataset, nunca visto durante el entrenamiento ni la validación**), proporciona una visión completa del comportamiento del modelo más allá de las métricas escalares.

6.3.1. Distribución de Errores: Centrado en Cero

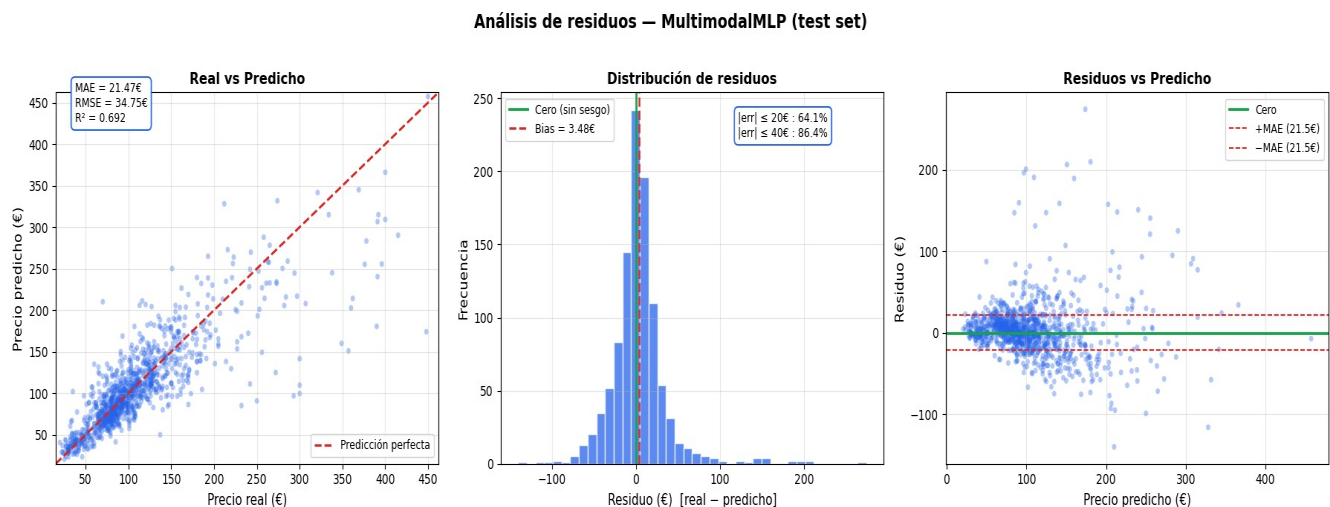


Figura 6.3.1. Análisis residuos RRNN multimodal

El histograma de residuos confirma que los errores se distribuyen de forma **aproximadamente simétrica y centrada en 0**.

Esta propiedad es fundamental: un bias próximo a cero indica que el modelo **no sobreestima ni subestima sistemáticamente** los precios de mercado. Si el bias fuera positivo, significaría que la red tiende a infravalorar los apartamentos; si fuera negativo, que los sobreestima. Ambos casos generarían un error sistemático que el inversor debería corregir manualmente, reduciendo el valor práctico del sistema.

La distribución presenta la forma de campana esperada con colas asimétricas, lo que es coherente con la distribución del precio objetivo: el modelo comete errores más grandes en los apartamentos del extremo superior del rango de precios (150–450€/noche), donde la muestra es más escasa y la variabilidad del mercado más alta.

6.3.2. Concentración del Error

El análisis cuantil de los residuos sobre el test set confirma que la distribución del error es favorable para el uso práctico del sistema:

- **~64% de las predicciones** tienen un error absoluto inferior o igual a 20€
- **~86.4% de las predicciones** tienen un error absoluto inferior o igual a 40€

Esto significa que para más de la mitad de los apartamentos del mercado malagueño, el sistema produce estimaciones con una precisión mejor que $\pm 20\text{€/noche}$ — suficiente para tomar decisiones de fijación de precio con confianza.

6.3.3. Heterocedasticidad: El Patrón de los Precios Altos

El gráfico de residuos frente a valores predichos revela un patrón de **heterocedasticidad moderada**: la dispersión de los errores crece conforme aumenta el precio predicho. Para apartamentos en el rango 18–100€/noche (el segmento central del mercado), los residuos se mantienen compactos entorno a la banda $\pm \text{MAE}$. Para el segmento 150–400€/noche, la dispersión se amplía.

Este comportamiento no es un defecto del modelo sino una consecuencia directa de la estructura del mercado. Los apartamentos de precio alto son intrínsecamente más heterogéneos: un ático en el Centro Histórico y una villa en Churriana pueden tener el mismo precio predicho basado en características físicas similares, pero sus diferencias de mercado — historial de reservas, calidad de la fotografía, valoraciones acumuladas — generan residuos mayores. La señal visual de ResNet34 captura parte de esta diferencia, pero no la totalidad.

6.4. Interpretabilidad: El Factor Estético

6.4.1. Qué Ha Aprendido la Red Visual

Los 512 embeddings extraídos por la capa avgpool de ResNet34 no son interpretables individualmente, pero su comportamiento agregado revela patrones económicamente coherentes. Durante el proceso de fine-tuning (capas layer3 y layer4 desbloqueadas), la red ajustó sus filtros de alto nivel para responder a características de los interiores que el mercado de alquiler vacacional valora:

- **Luminosidad global:** las fotografías con mayor cantidad de luz natural (ventanas amplias, tonos claros) tienden a generar embeddings que empujan la predicción al alza respecto al precio base tabular.
- **Densidad y modernidad del mobiliario:** espacios con mobiliario contemporáneo y bien distribuido frente a habitaciones vacías o con mobiliario anticuado.
- **Acabados de calidad:** la red detecta diferencias de textura entre suelos de parqué o microcemento y suelos de baldosa básica, entre paredes pintadas y paredes con gotele.

La consecuencia práctica es que la red ha aprendido a identificar que **una fotografía luminosa con mobiliario moderno actúa como multiplicador de precio sobre el valor base tabular**. Un apartamento de 3 habitaciones en La Malagueta con datos tabulares que justificarían un precio de 85€/noche puede obtener una predicción final de 105€/noche si su fotografía comunica calidad visual, y 75€/noche si la fotografía es oscura o está desactualizada.

6.4.2. Implicación para el Cálculo de ROI

Esta capacidad de la red visual tiene una implicación directa para la propuesta de valor central de SmartHost Analytics: **la cuantificación del ROI de una reforma estética**.

Si un inversor dispone de un apartamento cuya fotografía actual genera un multiplicador visual negativo respecto al precio base, el sistema puede simular el impacto económico de una mejora fotográfica o de interiorismo:

Precio actual (foto antigua) → Sistema predice 85€/noche

Precio simulado (foto reformada) → Sistema predice 108€/noche

Diferencia → +23€/noche
Ocupación estimada → 180 noches/año
Incremento de ingresos estimado → +4.140€/año
Coste de reforma estética → ~3.500€
ROI de la reforma → < 12 meses

Esta simulación, imposible con cualquier modelo tabular puro, es la razón técnica y de negocio que justifica toda la arquitectura multimodal del proyecto. El sistema no solo tasa: **convierte la intuición estética del inversor en una estrategia cuantificada de datos.**

7. DESPLIEGUE Y ARQUITECTURA DEL SISTEMA

7.1. Filosofía de Diseño: Arquitectura Desacoplada (Client-Server)

Para cumplir con los estándares de la industria del software y los criterios de evaluación del proyecto, el sistema de tasación inteligente se ha diseñado bajo una **arquitectura desacoplada en dos capas (Microservicios)**. Se descartó la integración directa de los pesos del modelo dentro de la interfaz gráfica por tres motivos de peso:

1. **Aislamiento de Entornos:** La interfaz de usuario no necesita importar librerías pesadas de cálculo científico como PyTorch o CUDA, reduciendo el tamaño del contenedor del cliente.
2. **Seguridad de Propiedad Intelectual:** Los pesos de las redes neuronales (.pt) y la receta de preprocesamiento (.pkl) residen exclusivamente en el servidor backend seguro, quedando protegidos contra ingeniería inversa en el lado del cliente.
3. **Escalabilidad:** Permite actualizar de forma independiente el modelo predictivo en el servidor (por ejemplo, reentrenar la red multimodal) sin necesidad de modificar o reinstalar la aplicación web de los usuarios.

7.2. Backend (API RESTful con FastAPI)

El núcleo operativo del sistema se aloja en un servidor web desarrollado con el *framework* **FastAPI**. Se seleccionó esta tecnología debido a su alto rendimiento basado en operaciones asíncronas (async/await), su validación de tipos en tiempo de ejecución mediante la librería **Pydantic** y su capacidad nativa para autogenerar documentación interactiva bajo el estándar OpenAPI (Swagger).

7.2.1. Inicialización y Carga de Modelos en Memoria

Al arrancar el servidor, se ejecuta un proceso de inicialización estática que carga en memoria VRAM/RAM los artefactos ganadores de las fases previas para evitar la latencia que supondría leer los archivos en cada petición HTTP:

- El pipeline de transformación tabular entrenado (preprocessor_V3.pkl) mediante `jllib.load()`.
- El extractor de características visuales (la red backbone **ResNet34** con sus capas convolucionales congeladas).

- El clasificador densificado multimodal entrenado (airbnb_mlp_PriceCAPV3_NoLog.pt) instanciado mediante `torch.load()` y configurado explícitamente en modo de inferencia pura con la directiva `model.eval()` y el gestor de contexto `torch.no_grad()`.

7.2.2. Diseño de Endpoints y Flujo de Inferencia

La API expone un endpoint principal configurado mediante el método HTTP POST en la ruta `/predict`. Dado que la red neuronal requiere entradas mixtas (un JSON con variables tabulares y un archivo binario de imagen), el endpoint está diseñado para consumir solicitudes de tipo `multipart/form-data`.

Cuando el servidor recibe una petición, procesa la información de forma estrictamente secuencial a través del siguiente conducto de datos (*pipeline*):

1. **Validación Tabular:** Los datos de entrada del apartamento (habitaciones, baños, barrio, amenities) se estructuran en un esquema de Pydantic que valida los tipos de datos y asigna valores por defecto ante omisiones.
2. **Transformación de Características:** Los datos validados se envían al `ColumnTransformer` que imputa nulos con la mediana aprendida en el entrenamiento, realiza el escalado estandarizado de los números y ejecuta el *One-Hot Encoding* de los textos.
3. **Procesamiento de Imagen:** El archivo binario de la foto se lee mediante un objeto `UploadFile`. Se somete de inmediato a las transformaciones geométricas idénticas al entrenamiento: reducción del lado menor a 256 píxeles, recorte central cuadrado de 224 x 224 y normalización de canales RGB.
4. **Fusión y Forward Pass:** La imagen procesada pasa por la `ResNet34` para extraer el vector de características de 512 dimensiones. Este vector se concatena con el vector tabular de 35 dimensiones, formando la entrada multimodal de 547 características. La red final ejecuta el pase hacia adelante (*forward pass*) y devuelve la predicción final en un JSON plano con la estructura `{"predicted_price": valor}`.

7.3. Frontend (Interfaz Gráfica con Streamlit)

Para dar una capa interactiva de nivel profesional y amigable de cara al inversor inmobiliario, se implementó una aplicación web nativa en Python utilizando el *framework* **Streamlit**. Esta interfaz actúa como un cliente puro encargándose de la captura de inputs, la comunicación asíncrona con el backend y la renderización visual de las métricas de negocio.

7.3.1. Arquitectura de la Interfaz de Usuario

El diseño visual se ha estructurado de manera ejecutiva en tres bloques funcionales bien diferenciados:

- **Panel Lateral de Características (Sidebar):** Contiene los elementos interactivos de entrada de datos. El usuario selecciona mediante menús desplegables (`st.selectbox`) el barrio oficial de Málaga y el tipo de alojamiento. Mediante selectores numéricos (`st.number_input`) indica la capacidad de huéspedes, número de camas y baños. Finalmente, un listado de casillas de verificación (`st.checkbox`) permite marcar de manera ágil los servicios extra de la vivienda (Aire acondicionado, WiFi, Piscina, etc.).
- **Módulo de Carga Visual:** Situado en el cuerpo central de la página, implementa un componente de arrastrar y soltar archivos (`st.file_uploader`) configurado exclusivamente para extensiones `.png`, `.jpg` y `.jpeg`. Al cargar el archivo, la app renderiza la imagen en tiempo real (`st.image`) para dar un feedback visual inmediato al usuario.
- **Consumo de la API:** Al pulsar el botón "Calcular Tasación", la interfaz empaqueta los inputs del formulario en un diccionario Python y la imagen en formato binario. Utilizando la librería `requests`, abre una conexión HTTP POST dirigida al backend de FastAPI. Si el servidor responde correctamente (Código 200), extrae el valor devuelto para inyectarlo en el simulador financiero.

Datos del apartamento

Foto de portada

Sube la foto actual del piso

Drag and drop file here

Limit 200MB per file • JPG, JPEG, P...

Browse files

Datos del anfitrión

☐ Soy anfitrión nuevo / perfil inversor

Tiempo de respuesta

within an hour

Tasa de respuesta

0.95

☒ ¿Es Superhost?

Ubicación

Dirección del apartamento

Ej: Calle Larios 5, Málaga

Tasación del apartamento

Predictor de precios para apartamentos turísticos en Málaga

Análisis completado

Precio por noche predicho

Precio Base (mercado)

108.54€

Precio Visual (con foto actual)

117.87€

+ 9.33€ por la foto

Impacto de la foto

8.6%

Estimación de ingresos anuales

Días ocupados/año

126.0 días

Ingresos base/año

13,676€

Ingresos con foto actual/año

14,852€

+ 1,176€ vs base

Deploy

Características

Tipo de alojamiento

Entire home/apt

Capacidad (personas)

4

Habitaciones

2

Camas

2

Baños

1,00

Servicios

☒ Cocina básica

☒ Calefacción

☒ TV

☐ Congelador

☒ Aire acond.

☒ Cafetera

☒ Lavadora

☐ Balcón/terraza

☒ Cocina completa

☒ Microondas

Valoraciones

Reseñas por mes

1,50

Reseñas últimos 12 meses

10

Valoración global

4.70

Precisión del anuncio

4.70

Limpieza

4.70

Ubicación

4.80

Relación calidad/precio

4.60

Calcular precio y ROI



Conjunto Figuras 7.3.1. Imágenes resultado Frontend

7.3.2. Lógica de Negocio: El Simulador de ROI Financiero

La verdadera innovación comercial de la interfaz radica en la traducción de las métricas matemáticas de Deep Learning en KPI financieros comprensibles para un inversor:

- **Cálculo de Ocupación Anual Estimada:** La aplicación extrae la variable `reviews_per_month` del histórico de la zona y aplica un algoritmo corrector multiplicándolo por dos (asumiendo que de media solo el 50% de los huéspedes reales escriben una reseña en la plataforma) y por la estancia media registrada en la ciudad de Málaga, acotando el resultado con un techo lógico de 365 días al año.
- **Proyección de Ingresos Brutos:** Multiplica los días estimados de ocupación anual por el precio por noche calculado dinámicamente por la red neuronal multimodal.
- **Cálculo de Retorno de Inversión (ROI):** La interfaz habilita una sección donde el usuario puede introducir el coste económico que le supondría llevar a cabo una reforma de interiorismo (por ejemplo: "Inversión en mobiliario: 4.000 €"). Al subir una foto mejorada o renderizada de la habitación tras el proyecto estético, el sistema calcula la diferencia entre los ingresos brutos del estado original y el estado reformado, mostrando al inversor cuántos meses tardará exactamente en amortizar cada euro invertido en la obra antes de realizar el gasto real.

8. DISCUSIÓN Y CONCLUSIONES

8.1. Resumen de los Hitos Logrados

El desarrollo e implementación de **SmartHost Analytics** ha permitido validar de forma empírica y científica la viabilidad de utilizar Inteligencia Artificial multimodal para la optimización de activos inmobiliarios turísticos en la ciudad de Málaga. A lo largo del ciclo de vida del proyecto, se han alcanzado con éxito hitos de ingeniería de datos y modelado avanzado que se resumen a continuación:

- **Construcción y Enriquecimiento de un Dataset Robusto:** Se consolidó una base de datos de 5.667 apartamentos turísticos mediante la extracción programática y limpieza profunda de fuentes de datos masivas. La implementación de ingeniería de características (*Feature Engineering*) permitió inyectar variables de altísimo valor estratégico, como el cálculo de distancias exactas a puntos críticos mediante la fórmula de *Haversine* (centro histórico y playas), ratios de hacinamiento por estancia y puntuaciones ponderadas de *amenities*.
- **Establecimiento de un Baseline Tabular Sólido:** El diseño de una red neuronal densa (MLP) en PyTorch demostró que las variables físicas y geográficas explican de forma aislada aproximadamente el 57% - 59% de la varianza de los precios de mercado, arrojando un error medio base (MAE) de 24,84 € por noche. Esta fase validó la decisión arquitectónica de aplicar un límite técnico (*Price Cap*) de 500 € para estabilizar el gradiente.
- **Pipeline de Procesamiento Visual Escalable:** Se desarrolló un script autónomo de procesamiento intermedio capaz de homogeneizar miles de imágenes a un estándar estricto de 224 X 224 píxeles mediante un algoritmo de escalado proporcional y recorte central (*CenterCrop*), garantizando la inmunidad del sistema ante archivos truncados o resoluciones anómalas.
- **Éxito en la Fusión Multimodal:** El mayor hito técnico del proyecto consistió en el acoplamiento de los *embeddings* estéticos de 512 dimensiones extraídos mediante aprendizaje por transferencia (*Transfer Learning* con ResNet34) con el vector tabular. La red combinada alcanzó un récord de rendimiento con un **R² de 0.692** y redujo el error predictivo a un **Val MAE de 21,47 €/noche**. Este incremento del

+10% en la capacidad explicativa del modelo constituye la prueba matemática de que el atractivo visual actúa como un modulador directo sobre el valor monetario de un inmueble.

- **Despliegue Orientado a Producción:** Se logró desacoplar con éxito la lógica computacional de la experiencia de usuario mediante una arquitectura de microservicios. El backend, implementado con FastAPI, expone modelos optimizados listos para inferencia asíncrona, mientras que el frontend interactivo en Streamlit proporciona a los inversores una herramienta intuitiva capaz de calcular el Retorno de Inversión (ROI) proyectado de una reforma estética antes de ejecutar el gasto de capital.

8.2. Limitaciones Encontradas

De acuerdo con las buenas prácticas de la investigación científica y la honestidad técnica exigida en la evaluación, es imperativo señalar las restricciones y desafíos intrínsecos que limitan el alcance del sistema actual:

- **Perspectiva Unifocal de la Imagen de Portada:** El modelo visual se alimenta exclusivamente de la fotografía principal del anuncio de Airbnb. Aunque esta imagen es el principal motor de conversión de clics en plataformas digitales, introduce un sesgo de información: la red es incapaz de evaluar si la alta calidad estética del salón se mantiene en estancias críticas como los cuartos de baño o la cocina, o si el anuncio recurre a técnicas fotográficas engañosas (ángulos hiperamplios o sobreexposición lumínica artificial, imágenes de paisajes y no del apartamento).
- **Techo Estadístico por Irracionalidad del Mercado:** El mercado vacacional inmobiliario e inmobiliario en general contiene un componente de varianza que ninguna inteligencia artificial puede modelar por completo. Factores humanos impredecibles, como estrategias de precios erróneas o emocionales por parte de anfitriones particulares, o urgencias de liquidez, provocan que propiedades idénticas presenten tarifas muy dispares en el mundo real, estableciendo un límite natural a la precisión del R^2 .
- **Pérdida de Información en Propiedades de Ultra-Lujo:** La necesidad de aplicar un filtro de corte (*Price Cap*) a 500 € para evitar la distorsión del aprendizaje causada por valores atípicos (*outliers*) provoca que el sistema actual quede inhabilitado para tasar de forma correcta villas exclusivas o apartamentos

residenciales premium que operen legítimamente por encima de dicho umbral (aunque la cantidad de apartamentos de este estilo es residual completamente).

- **Dependencia de la Vigencia de Enlaces:** El proceso de *web scraping* reveló que aproximadamente un 4% de las URLs de imágenes proporcionadas por los repositorios históricos estaban rotas o redirigían a servidores inactivos, obligando a purgar registros y demostrando la fragilidad de depender de almacenamiento externo a largo plazo.
- **Naturaleza Consultiva y Comportamiento No Lineal del Modelo Multimodal:** Durante las fases de pruebas iterativas del sistema, se ha observado que la introducción de variables discretas (como añadir una *amenity* específica) o la sustitución de la imagen por una de calidad estética teóricamente superior no siempre se traduce en un incremento lineal o predecible en el precio por noche final. Este fenómeno no constituye un defecto del software, sino que es una consecuencia directa de la arquitectura de *Deep Learning* seleccionada. Al fusionar los vectores latentes (tabulares y visuales), la super-red neuronal evalúa el apartamento de forma holística y no lineal. Esto significa que el impacto de una reforma estética está fuertemente condicionado por el contexto del inmueble (su ubicación geográfica o su capacidad). Por lo tanto, **SmartHost Analytics debe ser comprendido actualmente como una herramienta de carácter estrictamente consultivo y orientativo**, diseñada para guiar la toma de decisiones estratégicas, y no como un oráculo de tasación estricta al pie de la letra. El sistema marca tendencias y aproximaciones de mercado robustas, pero la fijación de precios final sigue requiriendo el criterio último del inversor humano.

8.3. Trabajo Futuro y Escalabilidad

Para garantizar el valor y la expansión comercial de SmartHost Analytics a largo plazo, se proponen las siguientes líneas de desarrollo tecnológico orientadas a la escalabilidad del sistema:

- **Arquitectura de Entrada Multi-Imagen:** Se plantea evolucionar la rama de visión artificial para que acepte un carrusel o galería completa de fotografías por inmueble.
- **Escalabilidad Geográfica Horizontal:** El *pipeline* de preprocesamiento, escalado y codificación categórica se ha diseñado de manera modular, lo que facilita su exportación inmediata a otras capitales nacionales con alta densidad de turismo

vacacional (como Madrid, Barcelona, Valencia o Sevilla). Únicamente se requeriría adaptar las coordenadas base para el cálculo de distancias de Haversine o restringir variables propias de la ciudad como distancia a la playa (propio de ciudades costeras) y reentrenar la última capa de la red multimodal con el histórico de precios local de cada urbe.

- **Integración de Series Temporales y Dynamic Pricing:** El modelo actual ofrece una tasación estática orientada a la toma de decisiones de inversión y reforma. El siguiente paso evolutivo consiste en acoplar modelos predictivos de series temporales (como arquitecturas LSTM o Prophet) para capturar la estacionalidad del mercado turístico malagueño, modulando el precio final según picos de demanda específicos (Semana Santa, Feria de Málaga o temporada estival).
- **Análisis de Sentimiento Mediante Procesamiento de Lenguaje Natural (NLP):** Incorporar modelos de lenguaje avanzados (LLMs o transformadores BERT especializados) para procesar el texto libre en variables como (description y neighborhood_overview) junto con los comentarios históricos de los usuarios. Extraer el sentimiento semántico y la reputación del anfitrión aportaría una tercera rama de datos a la red neuronal, consolidando un modelo híbrido Tabular-Visual-Textual de máxima fidelidad predictiva.
- **Optimización de la Sensibilidad del Modelo y Requerimientos Computacionales:** Para transformar el carácter actualmente orientativo del sistema en un modelo de tasación de alta fidelidad ante micro-cambios (como evaluar el impacto exacto de renovar un mueble específico o añadir un servicio menor), es imperativo continuar puliendo la sensibilidad de la red. Este proceso de optimización continua exige expandir la fase de ajuste fino (*Fine-Tuning*) de la rama visual y explorar hiperparámetros en arquitecturas de fusión más profundas. Sin embargo, esta evolución plantea una barrera técnica crítica: el coste computacional. Para entrenar un modelo híbrido capaz de procesar estas sutilezas estéticas sin perder la generalización del mercado se requerirá una capacidad de computación notablemente más alta (clústeres de GPUs dedicadas y entornos de computación distribuida). Esta inversión en hardware será el pilar fundamental para escalar el proyecto desde su estado actual de prototipo funcional avanzado hacia una plataforma comercial masiva de alta precisión.

9. REFERENCIAS

Repositorio GitHub

[Repositorio GitHub](#)

Artículos Científicos y Arquitecturas de Deep Learning

- **He, K., Zhang, X., Ren, S., & Sun, J. (2016).** *Deep Residual Learning for Image Recognition*. En Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778). (Referencia base para la arquitectura **ResNet34** utilizada en la extracción de características visuales).
- **Sandler, M., Howard, A., Zhu, M., Bhm, A., & Chen, L. C. (2018).** *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. En Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4510-4520). (Referencia evaluada durante la fase de experimentación visual).
- **Balanc, M., et al. (2016).** *Multimodal Deep Learning*. (Referencia teórica general sobre la técnica de concatenación (*Late Fusion*) utilizada para combinar la rama tabular y la rama visual en la red final).

Frameworks y Herramientas de Desarrollo

- **Paszke, A., et al. (2019).** *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. En Advances in Neural Information Processing Systems 32 (NeurIPS 2019). Recuperado de: <https://pytorch.org/> (Framework principal utilizado para el entrenamiento de los modelos MLP y Multimodal).
- **Pedregosa, F., et al. (2011).** *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. (Librería empleada para el escalado de datos StandardScaler y codificación OneHotEncoder).
- **FastAPI (2024).** *FastAPI Documentation*. Recuperado de: <https://fastapi.tiangolo.com/> (Framework asíncrono utilizado para el desarrollo y despliegue de la API RESTful).
- **Streamlit (2024).** *Streamlit Documentation*. Recuperado de: <https://docs.streamlit.io/> (Framework utilizado para la construcción de la interfaz gráfica interactiva y el simulador de ROI).
- **McKinney, W. (2010).** *Data Structures for Statistical Computing in Python* (Pandas). En Proceedings of the 9th Python in Science Conference (pp. 51-56).

Fuente de Datos

- **Inside Airbnb (2025).** *Adding data to the debate*. Recuperado de: <http://insideairbnb.com/> (Fuente original de los datasets históricos de propiedades turísticas y *web scraping* de imágenes para la ciudad de Málaga).